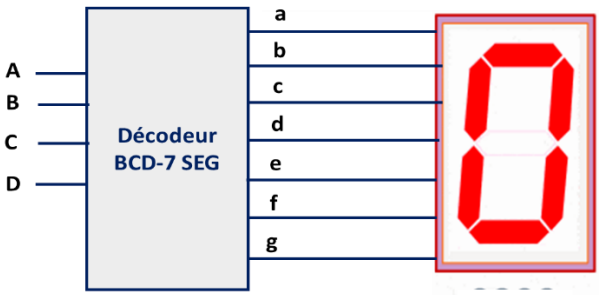
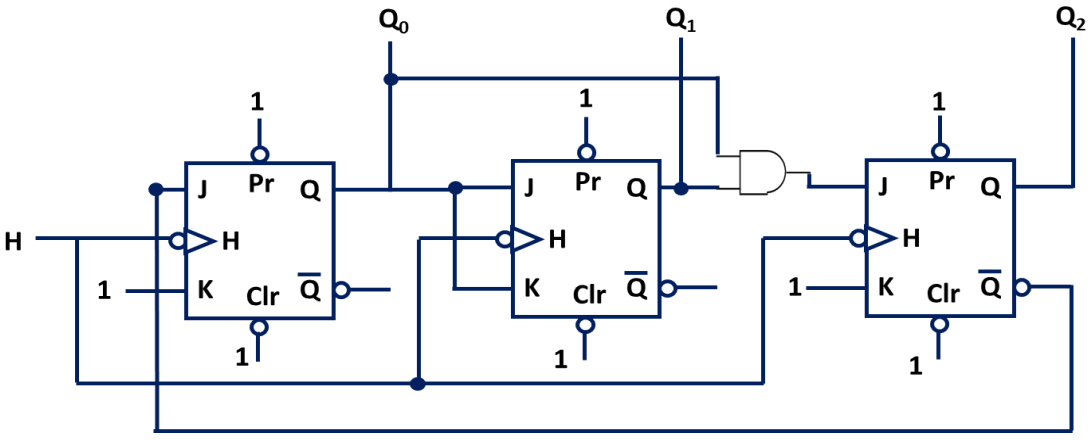
Université Ibn Zohr
Faculté des Sciences d'Agadir
Département de Physique

Filière : PE6 et TC-P4
Module : Electronique numérique



Notes de Cours :
Electronique numérique



Professeur responsable:

NOUREDDINE MAOUHOUB

Sommaire

Chapitre I : Systèmes de numération et codes……..3

Chapitre II : Algèbre de Boole et fonctions logiques…….18

Chapitre III : Circuits logiques combinatoires……31

Chapitre IV : Logique séquentielle et bascules……48

Chapitre V : Compteurs et registres……57

Chapitre I : Systèmes de numération et codes

1- Bases et systèmes de numération

1-1 Définition

La base d'un système de numération est le nombre de chiffres différents qu'utilise ce système de numération. Un nombre entier N de n chiffres est représenté en base b sous la forme suivante:

$$(N)_b = (a_{n-1}a_{n-2}.....a_1a_0)$$
$$\rightarrow a_i \in \{0,1,2.....(b-1)\}$$

Les a_i sont les symboles ou les chiffres. a_0 est le symbole de poids faible et a_{n-1} est le symbole de poids forts.

1-2 Exemple de bases de numération

1-2-1 Base décimale (b =10)

La base décimale est la plus utilisée dans le calcul courant. Le nombre de symboles utilisé par cette base est 10 chiffres appartenant à l'ensemble {0,1,2,3,4,5,6,7,8,9}.

Exemple : $N= (3256)_{10}$

6 est le chiffre de poids faible et 3 est le chiffre de poids forts.

1-2-2 Base binaire (b =2)

Cette base permet de représenter des nombres en utilisant deux symboles possibles {0,1}. Le chiffre binaire de cette base est appelé « bit ». Le système binaire est le plus utilisé en électronique numérique.

Exemple : $N = (11011)_2$

Le nombre N est représenté par cinq bits

1-2-3 Base octale (b = 8)

Le nombre de chiffres utilisé par cette base est huit chiffres appartenant à l'ensemble {0,1,2,3,4,5,6,7}.

Exemple : $N = (527)_8$

1-2-4 Base hexadécimale (b = 16)

Le nombre de chiffres utilisé par cette base est 16 chiffres appartenant à l'ensemble {0,1,2,3,4,5,6,7,8,9, A, B, C, D, E, F}. Le système de numération hexadécimal est utilisé souvent pour réduire la taille des nombres binaires.

N.B : Au-delà de 9, les chiffres de cette base sont représentés par des lettres (10 = A, 11 = B, 12 = C, 13 = D, 14 = E , 15 = F).

Exemple : N = $(1B6F)_{16}$

Le tableau ci-dessous résume les bases et systèmes de numération les plus utilisés.

	Système décimal	Système binaire	Système octal	Système hexadécimal
Base	b = 10	b = 2	b = 8	b= 16
Symboles	{0,1,2.....8,9}	{0,1} Le symbole est un « bit »	{0,1.....,7}	{0,1,.....9,A,B,C,D,E,F}
Exemple	$(1695)_{10}$	$(101101)_2$	$(257)_8$	$(62C)_{16}$

1-3 Représentation polynômiale d'un nombre entier en base b

Chaque nombre N écrit dans une base b est représenté sous la forme polynômiale suivante :

$$(N)_b = (a_{n-1}a_{n-2}.....a_1a_0) = \sum_{k=0}^{n-1} a_k b^k = a_{n-1}b^{n-1} + a_{n-2}b^{n-2} + ... + a_0b^0$$

ak est le symbole de rang k

b^k est la pondération associée à k

N.B : La somme des termes de ce polynôme est la valeur décimale du nombre N en base b.

Exemples :

- $(2365)_{10} = 2.10^3 + 3.10^2 + 6.10^1 + 5.10^0$
  5 est de rang 0, 6 est de rang 1, 3 est de rang 2 et 2 est de rang 3
- $(10111)_2 = 1.2^4 + 0.2^3 + 1.2^2 + 1.2^1 + 1.2^0 = 16 + 0 + 4 + 2 + 1 = (23)_{10}$
- $(6721)_8 = 6.8^3 + 7.8^2 + 2.8^1 + 1.8^0 = (3681)_{10}$
- $(A735)_{16} = 10.16^3 + 7.16^2 + 3.16^1 + 5.16^0 = (42805)_{10}$

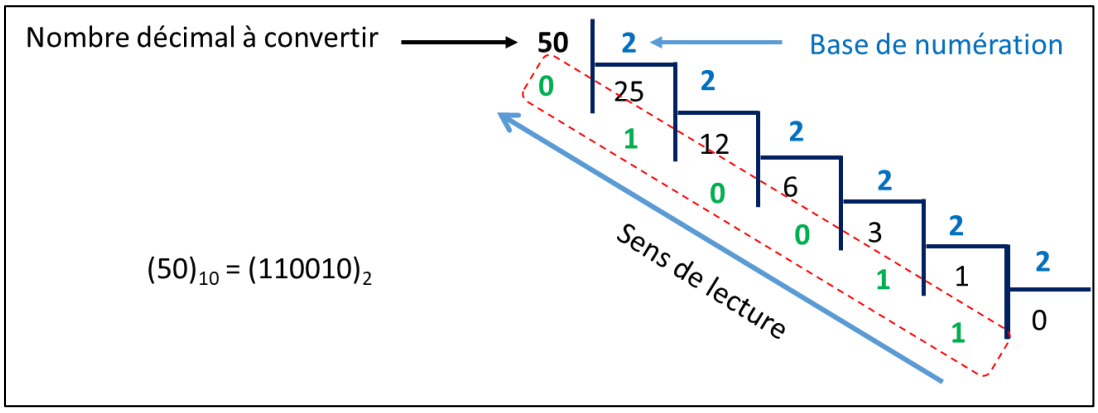
1-4 Changement de base et conversion

1-4-1 Conversion d'un nombre décimale en un nombre en base b

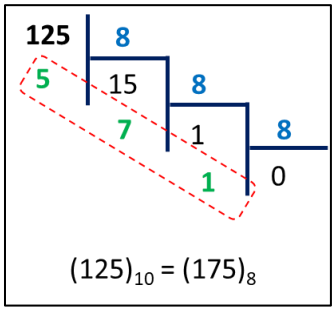
La conversion consiste à effectuer des divisions successives du nombre décimal N par la base b et conserver à chaque fois le reste, jusqu'à l'obtention d'un quotient nul. La lecture du nombre converti se fait à partir du dernier reste jusqu'à le premier reste.

Exemples :

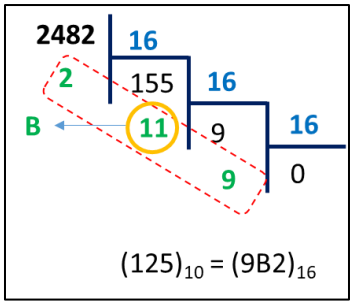
- Conversion du nombre décimal 50 en binaire



- Conversion du nombre décimal 125 en octal



- Conversion du nombre décimal 2482 en hexadécimal



Pour vérifier les valeurs converties, on peut recalculer la valeur trouvée en utilisant la forme polynômiale :

- $(175)_8 = 1.8^2 + 7.8^1 + 5.8^0 = 64 + 56 + 5 = (125)_{10}$
- $(9B2)_{16} = 9.16^2 + 11.16^1 + 2.16^0 = 2304 + 176 + 2 = (2482)_{10}$

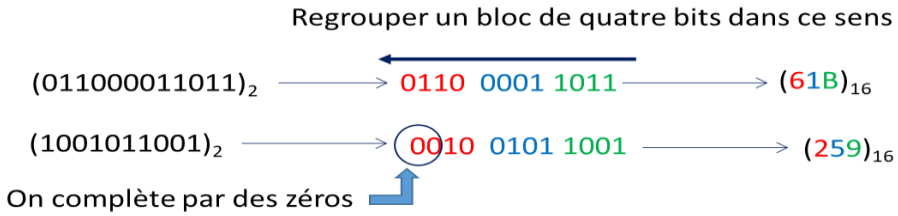
1-4-2 Conversion d'un nombre binaire en hexadécimal et vice-versa

La base de numération hexadécimale est une puissance de 2 : $16 = 2^4$.

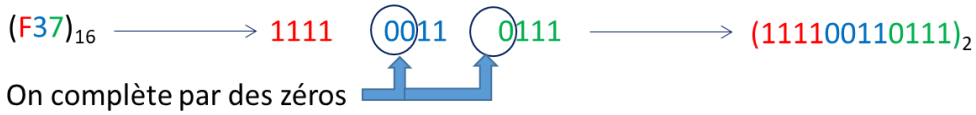
Pour convertir un nombre binaire en hexadécimal, on regroupe les bits par bloc de quatre en partant de la droite. Pour convertir un nombre hexadécimal en binaire, on convertit chaque chiffre en binaire en bloc de quatre bits.

Exemples :

- Convertir en hexadécimale les nombres binaires ci-dessous



- Convertir en binaire le nombres en hexadécimale ci-dessous



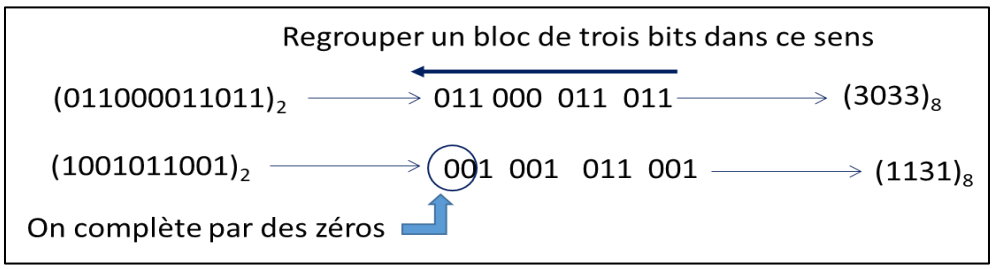
1-4-3 Conversion d'un nombre binaire en octal et vice-versa

La base de numération octale est une puissance de 2 : $8 = 2^3$

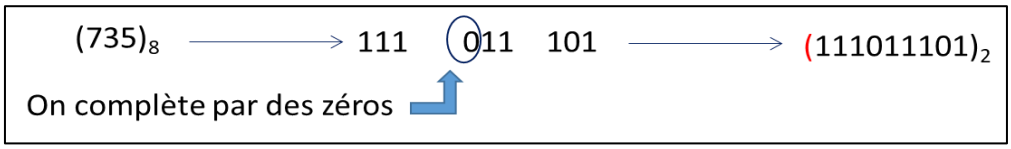
Pour convertir un nombre binaire en octal, on regroupe les bits par bloc de trois en partant de la droite. Pour convertir un nombre octal en binaire, on convertit chaque chiffre en binaire en bloc de trois bits.

Exemples :

- Convertir en octal les nombres binaires ci-dessous



- Convertir en binaire le nombre en octal ci-dessous



1-5 Représentation et conversion des nombres fractionnaires

1-5-1 Représentation polynômiale d'un nombre fractionnaire

La représentation de la partie fractionnaire de n chiffres en base b s'écrit sous la forme suivante :

$$(N)_b = (a_{-1}a_{-2}.....a_{-n}) = \sum_{k=1}^{n} a_{-k}b^{-k} = a_{-1}b^{-1} + a_{-2}b^{-2} + ... + a_{-n}b^{-n}$$
$$\rightarrow a_{-i} \in \{0,1,2.....(b-1)\}$$

N.B : La somme des termes de ce polynôme est la valeur décimale de la partie fractionnaire.

Exemples :

- $(0,536)_{10} = 5.10^{-1} + 3.10^{-2} + 6.10^{-3} = 0.5 + 0.03 + 0.006$
- $(0,1101)_2 = 1.2^{-1} + 1.2^{-2} + 0.2^{-3} + 1.2^{-4} = 0.5 + 0.25 + 0.0625 = (0.8125)_{10}$
- $(0,2C)_{16} = 2.16^{-1} + 12.16^{-2} = 0.125 + 0,00390625 = (0,12890625)_{10}$

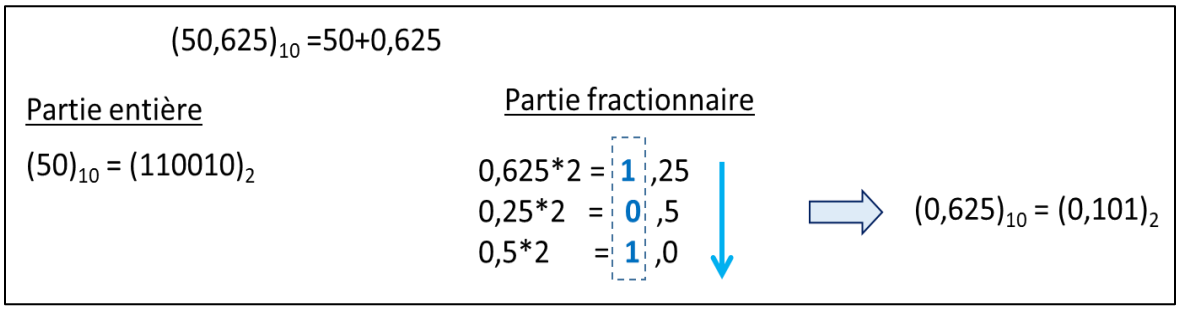
1-5 -2 Conversion d'un nombre décimal fractionnaire en un nombre en base b

La conversion de la partie entière se fait des divisions successives par la base b jusqu'à l'obtention d'un quotient nul (même procédure citée précédemment).

La conversion de la partie fractionnaire se fait par multiplication successives par la base b et conserver chaque fois la partie entière jusqu'à l'obtention d'une partie fractionnaire nulle ou d'une précision souhaitée. Le sens de lecture est du haut vers le bas.

Exemples :

- Convertir en binaire le nombre réel 50,625



On peut vérifier le résultat obtenu en utilisant la forme polynômiale de la partie fractionnaire:

$$(0,101)_2 = 1.2^{-1} + 0.2^{-2} + 1.2^{-3} = 0.5 + 0.125 = (0.625)_{10}$$

Donc: $(50,625)_{10} = (110010,101)_2$

- Convertir en binaire le nombre décimal réel 3,14

Partie entière $(3)_{10} = (11)_2$

Partie fractionnaire

0,14*2 = 0 ,28
0,28*2 = 0 ,56
0,56*2 = 1 ,12
0,12*2 = 0 ,24
0,24*2 = 0 ,48
0,48*2 = 0 ,96
0,96*2 = 1 ,92
0,92*2 = 1 ,84
.....

$(3,14)_{10} = (11,00100011...)_2$

N.B : - La partie fractionnaire peut être représenter par une suite infinie.
- Le développement s'arrête jusqu'à la précision souhaitée

On vérifie le résultat en utilisant la forme polynômiale des huit premiers bits :

$$0,00100011 = 0,125+ 0,0078125 + 0,00390625 = 0,13671875$$

- Convertir en octal le nombre décimal 0,732

0,732*8 = 5 , 856
0,856*8 = 6 , 848
0,848*8 = 6 , 784
0,784*8 = 6 , 272
0,272*8 = 2 , 176

$(0,732)_{10} = (0,56662...)_8$

On vérifie le résultat par la forme polynômiale :

$$(0,56662)_8 = 5.8^{-1} + 6.8^{-2} + 6.8^{-3} + 6.8^{-4} + 2.8^{-5} = 0.625 + 0.063750 + 0.011718750 = 0.730468750$$

1-5-3 Conversion d'un nombre fractionnaire binaire en hexadécimal et vice-versa

A partir de la virgule, on regroupe les bits par bloc de quatre en partant vers la gauche pour la partie entière et vers la droite pour la partie fractionnaire. Pour convertir un nombre fractionnaire hexadécimal en binaire, on fait l'opération inverse.

Exemples :

- Convertir en hexadécimal le nombre binaire suivant :

$(1001110010111010,0111)_2 = (9CBA,7)_{16}$

$(1001\ \ 1100\ \ 1011\ \ 1010\ ,\ \ 0111)_2$

$(9\ \ \ \ C\ \ \ \ B\ \ \ \ A\ \ ,\ \ 7)_{16}$

- Convertir en binaire le nombre hexadécimal suivant :

$(A78,B32)_{16}$

(A 7 8 , B 3 2)$_{16}$

(1010 0111 1000 , 1011 0011 0010)$_2$

1-5- Conversion d'un nombre fractionnaire binaire en octal et vice-versa

A partir de la virgule, on regroupe les bits par bloc de trois en partant vers la gauche pour la partie entière et vers la droite pour la partie fractionnaire. Pour convertir un nombre fractionnaire octal en binaire, on fait l'opération inverse.

Exemples :

- Convertir en octal le nombre binaire suivant :

$(010111010,0111)_2 = (272,34)_8$

(010 111 010 , 011 100)$_2$

(2 7 2 , 3 4)$_8$

- Convertir en binaire le nombre octal suivant :

$(150,62)_8$

(1 5 0 , 6 2)$_8$

(001 101 000 , 110 010)$_2$

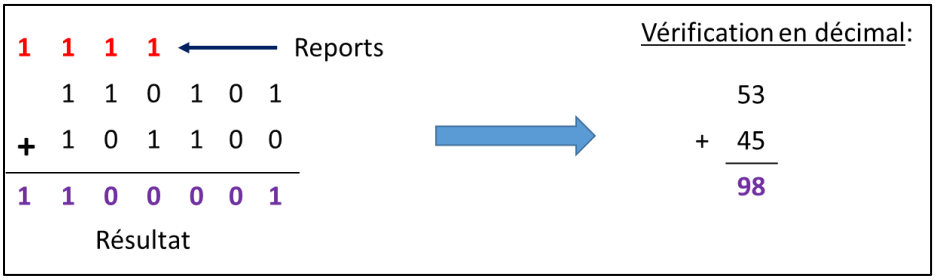
2- Opérations arithmétiques en binaire

2-1 Addition

La table d'addition en binaire est donnée par:

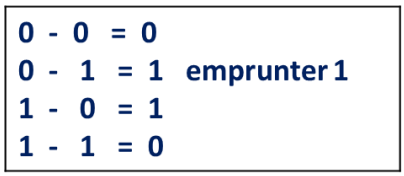
0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 0 et report de 1

Exemple :

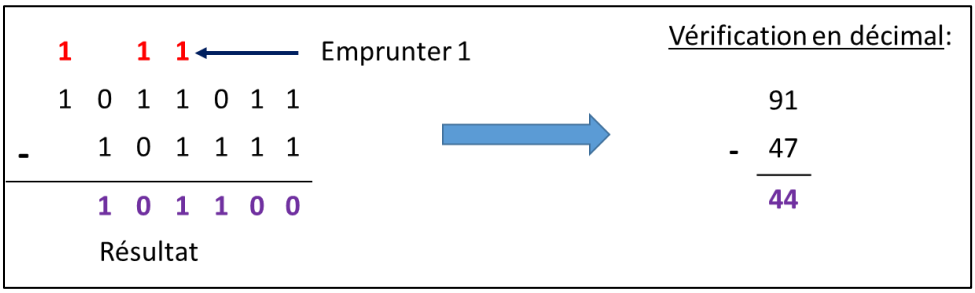


2-2 Soustraction

La table de soustraction en binaire est donnée par:

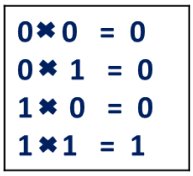


Exemple :

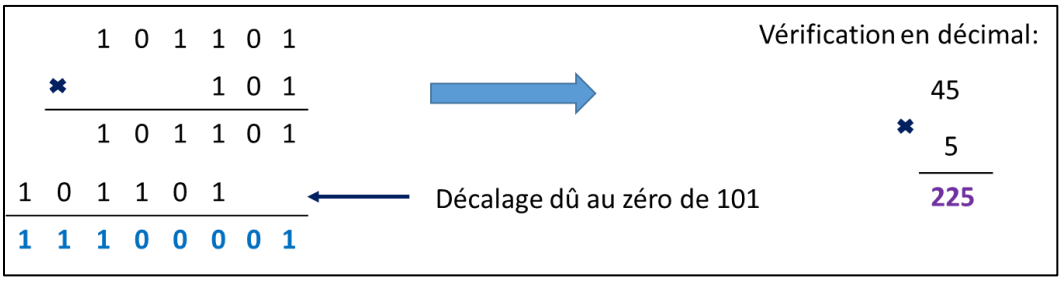


2-3 Multiplication

La table de multiplication en binaire est donnée par:



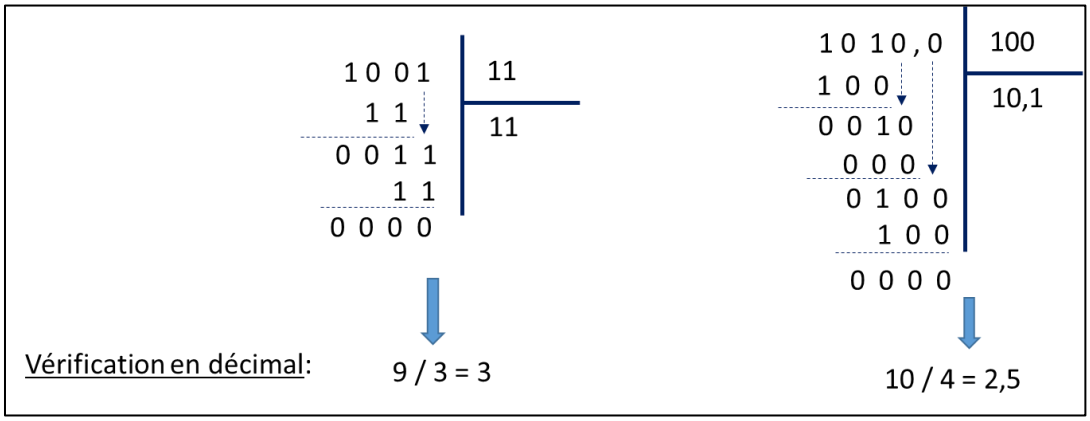
Exemple :



2-4 Division

La division d'un nombre binaire par un autre nombre binaire est identique à la division de deux nombres décimaux. La division en binaire est plus simple puisque pour déterminer combien de fois le diviseur entre dans le dividende, il n'y a que 2 possibilités 0 ou 1.

Exemples :



3- Capacité de la représentation en binaire

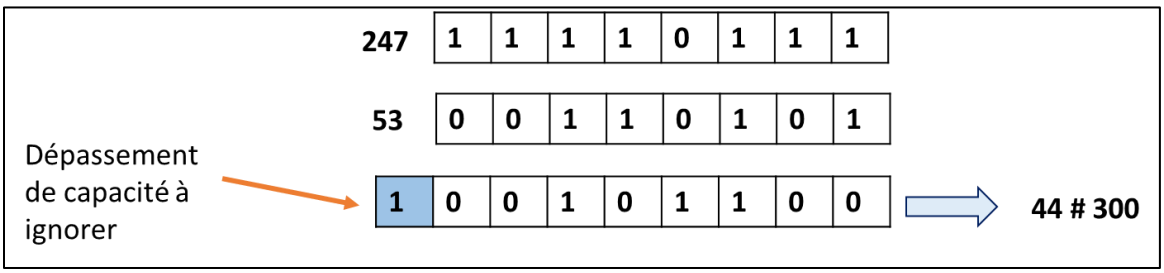
La limite du codage des nombres est due à la longueur du mot binaire nécessaire pour les coder. En binaire avec n bits, on ne peut représenter qu'un nombre naturel entre **0 et 2^n-1**, soit **2^n valeurs.**

Le tableau ci-dessous résume le nombre et la limites des valeurs possibles pour 4, 8 et 16 bits.

Bits	Nombre de valeurs	Limite de valeurs
4	16	0 à 15
8	256	0 à 255
16	65536	0 à 65535

En addition binaire de deux nombres codés sur n bits, le résultat n'est pas toujours possible, car on peut avoir un dépassement de capacité.

Exemple :



4- Complémentation en binaire

4-1 Complément à 1

Pour déterminer le complément à 1 d'un nombre binaire, il faut complémenter chaque bit: le 1 devient 0 et le 0 devient 1. Le complément à 1 est appelé aussi le complément restreint CR.

Exemples :

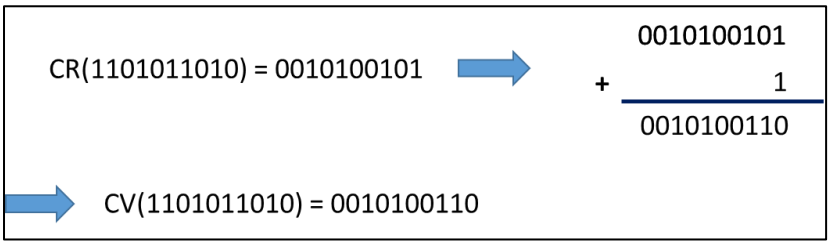
- CR(0110011) = (1001100)
- CR(011110) = (100001)

4-2 Complément à 2

Le complément à 2 est noté aussi le complément vrai CV. Pour déterminer ce complément, on cite deux méthodes.

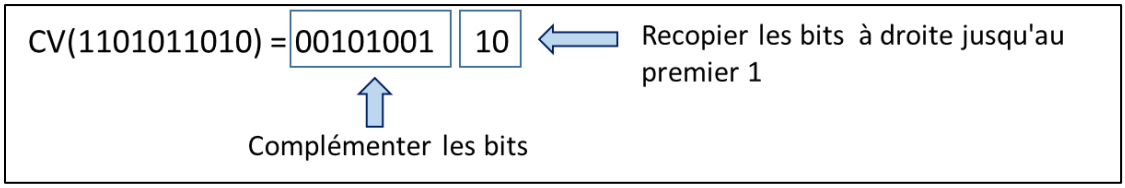
- **Méthode 1 :** Pour déterminer le complément à 2 d'un nombre binaire, il faut déterminer le complément à 1 de ce nombre et de lui ajouter 1.

Exemple : Déterminer le complément à 2 du nombre binaire: 1101011010



- **Méthode 2 :** Le complément à 2 d'un nombre binaire consiste à conserver tous les bits à droite jusqu'au premier 1 et de complémenter les autres bits.

Exemple : Déterminer le complément à 2 du nombre binaire: 1101011010



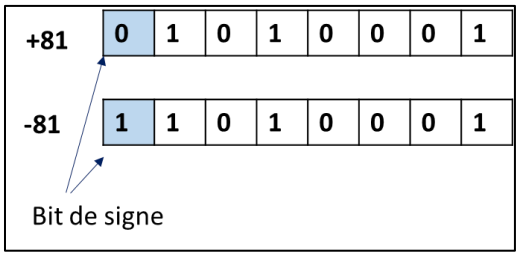
N.B : CV(CV(N)) = N

5- Représentation des nombres signés

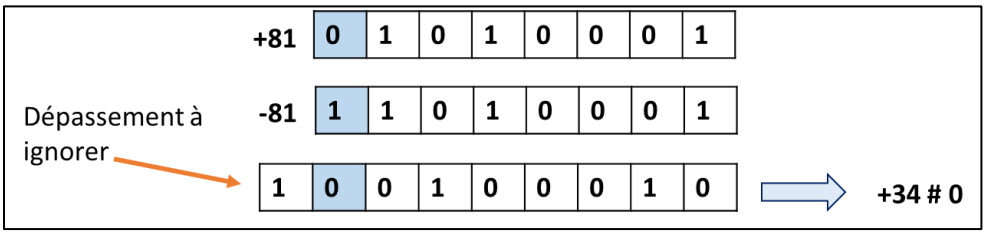
5-1 Représentation en valeur absolue

Cette représentation consiste à ajouter un bit à gauche pour le signe (0 pour le signe positif et 1 pour le signe négatif) et représenter sur les autres bits la valeur absolue du nombre. Cette représentation est rarement utilisée, puisqu'elle pose un problème en arithmétique binaire avec deux représentation du zéro (positive et négative).

Exemple : Représentation sur 8 bits



L'addition binaire de +81+(-81) ne donne pas zéro car il y a un dépassement de capacité (le résultat dépasse les 8 bits).

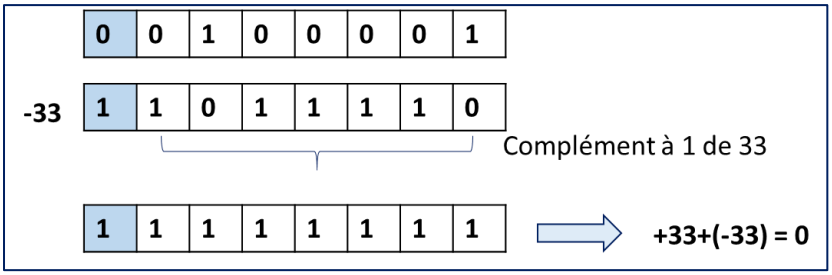


5-2 Représentation en complément à 1

Pour une représentation sur n bits, on ajoute un bit de signe dans le bit de poids fort (0 pour signe positif et 1 pour signe négatif).

Le nombre positif est représenté par la forme binaire naturelle sur n-1 bits et le nombre négatif est représenté par le complément à 2.

- Représenter en complément à 1 sur 8 bits les nombres +33 et -33 et effectuer l'addition en binaire +33+(-33)



Remarque :

Pour lire la valeur décimale d'un nombre binaire écrit en complément à 1, on observe le bit de poids fort et on procède comme suit :

- Si ce bit est 0, le nombre est positif et sa valeur décimale est calculer directement par la forme polynomiale.
  Exemple : soit le nombre **N = (01101)$_2$** écrit en complément à 1sur 5 bits. Ce nombre est positif et sa valeur décimale est 13, donc N = $(+13)_{10}$.
- Si ce bit est 1, le nombre est négatif. Dans ce cas, il faut calculer à nouveau le complément à 1 de ce nombre et déterminer par la suite sa valeur décimale.

Exemple : soit le nombre **N = (11101)$_2$** écrit en complément à 1 sur 5 bits. Ce nombre est négatif, donc on calcule son complément à 1 : (00010) et par la suite on calcule sa valeur décimale par la forme polynomiale qui est égale à 2. Enfin, on écrit : **N = (-2)$_{10}$**

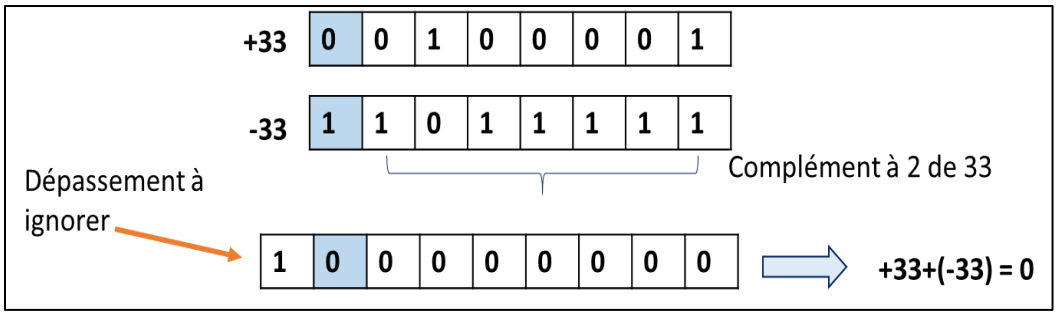
5-3 Représentation en complément à 2

Pour une représentation sur n bits, on ajoute un bit de signe dans le bit de poids fort (0 pour signe positif et 1 pour signe négatif).

Le nombre positif est représenté par la forme binaire naturelle sur n-1 bits et le nombre négatif est représenté par le complément à 2.

Exemples :

- Représenter en complément à 2 sur 8 bits les nombres +33 et -33 et effectuer l'addition en binaire +33+(-33)



Remarque :

Pour lire la valeur décimale d'un nombre binaire écrit en complément à 2, on observe le bit de poids fort et on procède comme suit :

- Si ce bit est 0, le nombre est positif et sa valeur décimale est calculer directement par la forme polynomiale.

Exemple : soit le nombre **N = (0101)$_2$** écrit en complément à 2sur 4 bits. Ce nombre est positif et sa valeur décimale est 5, donc **N = (+5)$_{10}$**.

- Si ce bit est 1, le nombre est négatif. Dans ce cas, il faut calculer à nouveau le complément à 1 de ce nombre et déterminer par la suite sa valeur décimale.

Exemple : soit le nombre **N = (1101)$_2$** écrit en complément à 2 sur 4 bits. Ce nombre est négatif, donc on calcule son complément à 2 : (0011) et par la suite on calcule sa valeur décimale qui est égale à 3. Enfin, on écrit : **N = (-3)$_{10}$**

6- Codage de l'information

6-1 Code binaire naturel

C'est un code dans lequel on exprime un nombre suivant le système de numération binaire. Ce code présente un inconvénient : il introduit des erreurs lors du codage des nombres variant de façon ordonnée. Entre deux codes successifs, plusieurs bits peuvent changer simultanément.

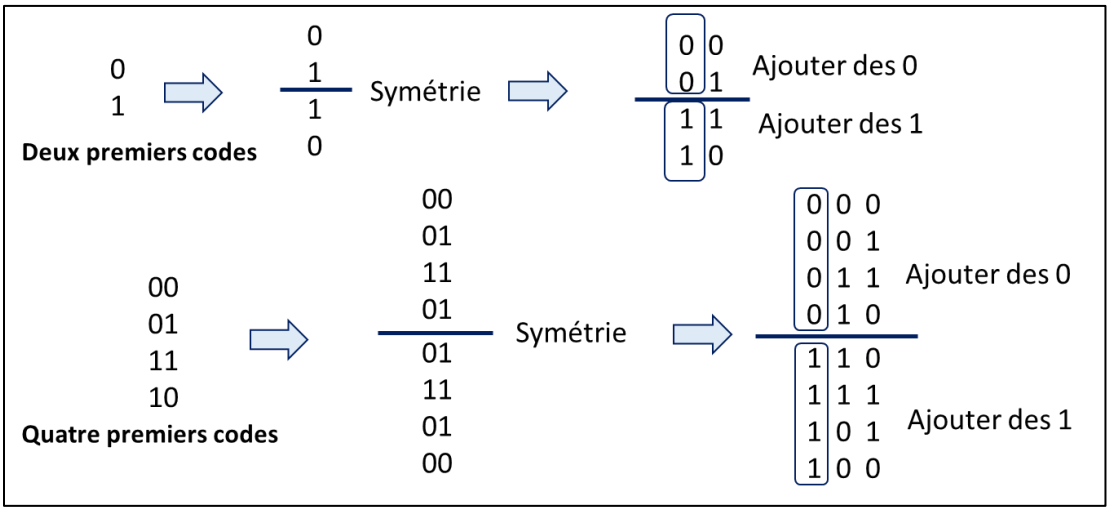
Le tableau ci-dessous donne un exemple de code binaire naturel des valeurs décimales allant de 0 à 15 et codées sur quatre bits.

Nombre décimal	Code binaire naturel			
	a_3	a_2	a_1	a_0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

Tableau 1.1 : Codes binaires naturels des chiffres 0 à 15

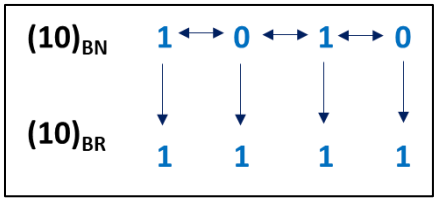
6-2 Code binaire réfléchi (code de Gray)

Ce code consiste à coder un nombre en binaire tel que à chaque augmentation d'une unité du chiffre décimal, un seul bit du nombre binaire équivalent change de valeur par rapport au nombre binaire précédant. Pour construire un tableau des codes binaires réfléchis, on utilise des réflexions successives selon le schéma suivant :



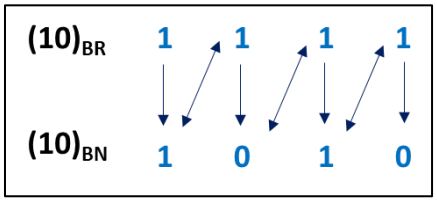
- **Conversion du binaire naturel en binaire réfléchi:** La méthode consiste à recopier le bit à gauche du nombre binaire naturel, et comparer le bit b_{n+1} et b_n, le bit b_r résultant du code binaire réfléchi vaut 0 si $b_{n+1} = b_n$ et vaut 1 si $b_{n+1} \# b_n$

Exemple :



- **Conversion du binaire réfléchi en binaire naturel** : La méthode consiste à recopier le bit à gauche du nombre binaire réfléchi, et comparer le bit b_{n+1} du binaire naturel et le bit b_n du binaire réfléchi. Le bit b_n résultant du code binaire naturel vaut 0 si $b_{n+1} = b_n$ et vaut 1 si $b_{n+1} \# b_n$

Exemple :



6-3 Code Décimal Codé Binaire (DCB)

Le code DCB (ou BCD: Binary Coded Decimal) consiste à coder chaque chiffre décimal (0 jusqu'à 9) sur quatre bits selon son équivalent binaire. Le tableau ci-dessous résume les dix chiffres décimaux ainsi que leur équivalent en DCB.

0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Tableau 1.2 : Codes BCD de 0 à 9

Exemple : Donne le code DCB du chiffre décimal 9708

$(9708)_{10} = (1001\ 0111\ 0000\ 1000)_{DCB} = (1001011100001000)_{DCB}$

6-4 Code alphanumérique: Code ASCII

Ce code est utilisé pour les échanges en informatique. Il permet de coder les 26 lettres de l'alphabet, les 10 chiffres, les signes de ponctuation et les caractères de contrôle.

USASCII code chart

$b_7 b_6 b_5$ →	0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1
Bits b_4 ↓	b_3	b_2	b_1	Column → / Row ↓	0	1	2	3	4	5	6	7
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	|
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

Tableau 1.3: Codes ASCI

Exemples :

- « K » = 1001011 en binaire
- « SP » (Espace)= 0100000 en binaire

Chapitre II : Algèbre de Boole et fonctions logiques

1- Algèbre de Boole

1-1 Variable logique

On l'appelle aussi variable booléenne ou variable binaire. C'est une variable qui ne peut prendre que deux états logiques: 0 ou 1.

Exemple :

La fermeture de l'interrupteur K (K = 1) permet d'allumer la lampe S (S = 1) et l'ouverture de K (K = 0) permet d'éteindre la lampe (S = 0).

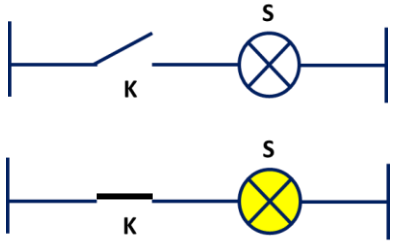


Figure 2.1 : Commande de l'allumage d'une lampe par un interrupteur K

1-2 Fonction logique

La fonction logique est une combinaison de variables logiques. Une fonction logique prend deux états logiques: 0 ou 1.

Exemple :

x et y sont deux variables logiques.

$$F = (x + y)(x + \bar{y})$$

1-3 Opérateurs logiques fondamentaux

- **Opérateur logique « OUI » (Egalité)**

La table de vérité de cet opérateur est donnée par :

A	A
0	0
1	1

- **Opérateur logique « NON » (Complémentation ou négation)**

Cet opérateur, appelé aussi « NOT » en anglais, permet d'inverser les opérateurs logiques, le « 0 » devient « 1 » et le « 1 » devient « 0 ». La table de vérité de cet opérateur est donnée par :

A	$\bar{A}$
0	1
1	0

- **Opérateur logique « OU » (Somme logique)**

Cet opérateur appelé aussi « OR » en anglais, permet de réaliser une somme logique des variables logiques. La table de vérité de cet opérateur est donnée par :

A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

- **Opérateur logique « ET » (Produit logique)**

Cet opérateur appelé aussi « AND » en anglais, permet de réaliser un produit logique des variables logiques. La table de vérité de cet opérateur est donnée par :

A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1

1-4 Différentes propriétés et théorèmes en algèbre de Boole

- **Propriétés des opérateurs logiques**

Le tableau suivant résume les différentes propriétés des opérateurs logiques. Ces propriétés seront utilisées pour simplifier les fonctions logiques.

	Somme logique	Produit logique
Commutativité	A+B=B+A	A.B=B.A
Associativité	(A+B)+C=A+(B+C)	(A.B).C=A.(B.C)
Distributivité	A.(B+C)=A.B+A.C	A+(B.C)=(A+B).(A+C)
Identité (élément neutre)	A+0=A	A.1=A
Complémentarité	$A+\overline{A}=1$	$A.\overline{A}=0$
Elément absorbant	A+1=1	A.0=0
Idempotence	A+A=A	A.A=A

Tableau 2.1 : Différentes propriétés logiques

- **Théorèmes**

Le tableau ci-dessous résume les théorèmes utilisés pour manipuler des fonctions logiques.

	Somme logique	Produit logique
Théorème de l'absorption	A+A.B=A	A.(A+B)=A
Théorème d'allègement	$A+\overline{A}B=A+B$	$A.(\overline{A}+B)=A.B$
Théorème de De Morgan	$\overline{A+B}=\overline{A}\,\overline{B}$	$\overline{AB}=\overline{A}+\overline{B}$
Involution	$A=\overline{\overline{A}}$	

Tableau 2.2 : Différents théorèmes utilisés dans l'algèbre de Boole

2- Portes logiques

Les portes logiques sont des circuits électroniques construites par des diodes ou des transistors bipolaires ou des transistors MOS. Ces circuits permettent de matérialiser des fonctions logiques. Dans cette partie, on va définir les différentes portes logiques utilisées en électronique numérique ainsi que leur symbole et quelques exemples de circuits logiques commerciaux fabriqués à base de transistors bipolaire (technologie TTL).

2-1 Portes logiques de base

- **Porte logique « NON » (NOT)**

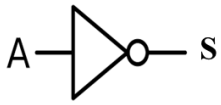
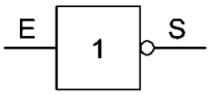
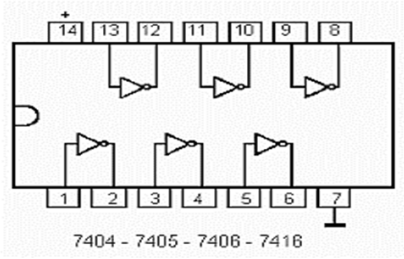
Symbole Américain (International)	Symbole Européen	Circuit intégré
		

Figure 2.2 : Symboles d'une porte NOT avec un exemple de son circuit intégré

Table de vérité et de fonctionnement

Table de vérité

A	S
0	**1**
1	**0**

Table de fonctionnement

A	S
V_b	$\mathbf{V_h}$
V_h	$\mathbf{V_b}$

V_b = 0V : Niveau bas
V_h = 5 V : Niveau haut

- **Porte logique « OU » (OR)**

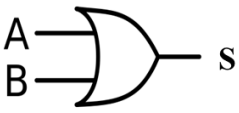
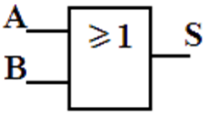
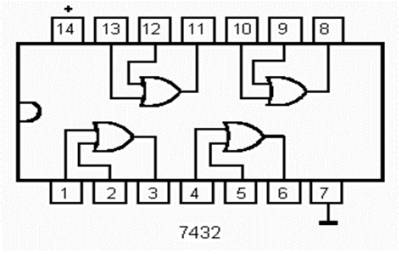
Symbole Américain (International)	Symbole Européen	Circuit intégré
		

Figure 2.3 : Symboles d'une porte OR avec un exemple de son circuit intégré

Table de vérité et de fonctionnement

Table de vérité

A	B	S
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**1**

Table de fonctionnement

A	B	S
V_b	V_b	$\mathbf{V_b}$
V_b	V_h	$\mathbf{V_h}$
V_h	V_b	$\mathbf{V_h}$
V_h	V_h	$\mathbf{V_h}$

V_b = 0V : Niveau bas
V_h = 5 V : Niveau haut

- **Porte logique « ET» (AND)**

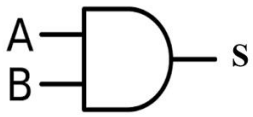
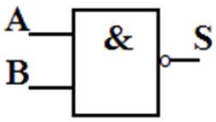
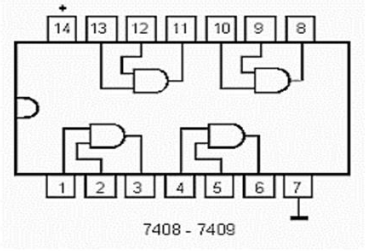
Symbole Américain (International)	Symbole Européen	Circuit intégré
		

Figure 2.4 : Symboles d'une porte AND avec un exemple de son circuit intégré

Table de vérité et de fonctionnement

Table de vérité

A	B	S
0	0	**0**
0	1	**0**
1	0	**0**
1	1	**1**

Table de fonctionnement

A	B	S
V_b	V_b	$\mathbf{V_b}$
V_b	V_h	$\mathbf{V_b}$
V_h	V_b	$\mathbf{V_b}$
V_h	V_h	$\mathbf{V_h}$

V_b = 0V : Niveau bas
V_h = 5 V : Niveau haut

2-2 Portes logiques « OU exclusif (XOR) » et « NON OU exclusif (XNOR) »

- **Porte logique XOR**

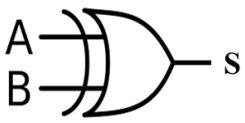
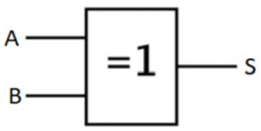
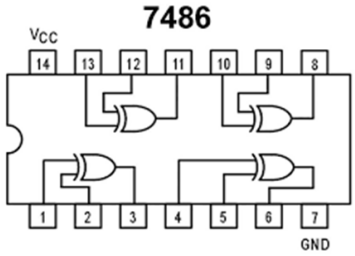
Symbole Américain (International)	Symbole Européen	Circuit intégré
		

Figure 2.5 : Symboles d'une porte XOR avec un exemple de son circuit intégré

Table de vérité et de fonctionnement

Table de vérité

A	B	S
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**0**

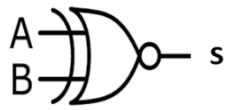
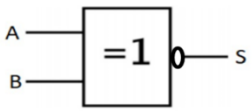
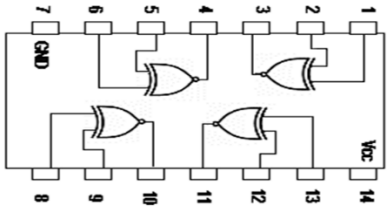
Table de fonctionnement

A	B	S
V_b	V_b	$\mathbf{V_b}$
V_b	V_h	$\mathbf{V_h}$
V_h	V_b	$\mathbf{V_h}$
V_h	V_h	$\mathbf{V_b}$

L'opérateur « ou exclusif » est définit par l'équation logique suivante :

$$S = A \oplus B = A.\overline{B} + \overline{A}.B$$

- **Porte logique XNOR**

Symbole Américain (International)	Symbole Européen	Circuit intégré
		

*

Figure 2.6 : Symboles d'une porte XNOR avec un exemple de son circuit intégré

Table de vérité et de fonctionnement

Table de vérité

A	B	S
0	0	**1**
0	1	**0**
1	0	**0**
1	1	**1**

Table de fonctionnement

A	B	S
V_b	V_b	$\mathbf{V_h}$
V_b	V_h	$\mathbf{V_b}$
V_h	V_b	$\mathbf{V_b}$
V_h	V_h	$\mathbf{V_h}$

L'opérateur « XNOR » est définit par l'équation logique suivante :

$$S = \overline{A \oplus B} = A.B + \overline{A}.\overline{B}$$

2-3 Portes logiques universelles

- **Porte logique « NON-ET (NAND) »**

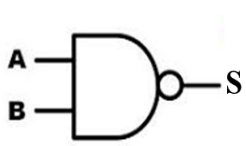
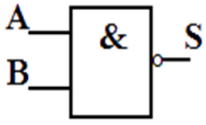
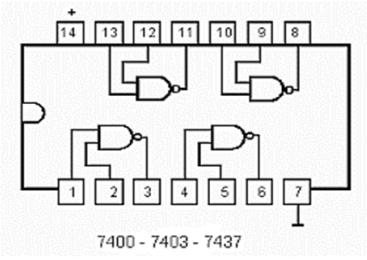
Symbole Américain (International)	Symbole Européen	Circuit intégré
		

Figure 2.7 : Symboles d'une porte NAND avec un exemple de son circuit intégré

Table de vérité et de fonctionnement

Table de vérité

A	B	$S=\overline{AB}$
0	0	**1**
0	1	**1**
1	0	**1**
1	1	**0**

Table de fonctionnement

A	B	S
V_b	V_b	$\mathbf{V_h}$
V_b	V_h	$\mathbf{V_h}$
V_h	V_b	$\mathbf{V_h}$
V_h	V_h	$\mathbf{V_b}$

- **Porte logique « NON-OU (NOR) »**

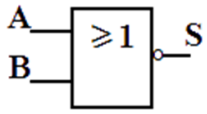
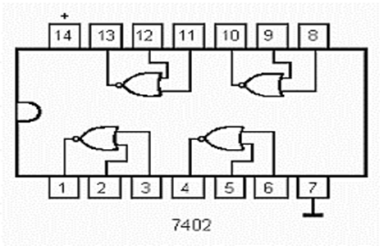
Symbole Américain (International)	Symbole Européen	Circuit intégré
		

Figure 2.8 : Symboles d'une porte NOR avec un exemple de son circuit intégré

Table de vérité et de fonctionnement

Table de vérité

A	B	
0	0	**1**
0	1	**0**
1	0	**0**
1	1	**0**

Table de fonctionnement

A	B	S
V_b	V_b	$\mathbf{V_h}$
V_b	V_h	$\mathbf{V_b}$
V_h	V_b	$\mathbf{V_b}$
V_h	V_h	$\mathbf{V_b}$

3- Représentation des fonctions logiques

Pour représenter une fonction logique, on utilise différentes représentations:

- Représentation algébrique
- Représentation par utilisation d'une table de vérité
- Représentation par utilisation d'un logigramme
- Représentation par utilisation du tableau de Karnaugh

3-1 Représentation algébrique

Une fonction logique est une combinaison de plusieurs variables logiques reliées par les différents opérateurs logiques ET, OU et NON. On peut représenter les fonctions logiques sous forme **Somme de Produits (SDP)** ou bien **Produit de Sommes (PDS).**

Exemples :

$F_1(a,b,c) = bc + ab\bar{c}$

$F_2(a,b,c) = (a+b)(\bar{b}+c)$

3-1-1 Formes canoniques

On dit que la forme algébrique d'une fonction logique est canonique si chacun des sommes ou des produits contient toutes les variables sous forme directe ou complémentaire.

- **Première forme canonique somme de produits (forme disjonctive):**

Dans le cas d'une forme canonique somme de produits, chacun des produits s'appelle **minterme**.

Exemple :

$$F(a,b,c) = abc + a\bar{b}c + ab\bar{c} + \bar{a}bc$$ ⟵ Exemple de minterme

- **Deuxième forme canonique produit de sommes (forme conjonctive):**

Dans le cas d'une forme canonique produit de sommes, chaque terme des sommes s'appelle **maxterme**.

Exemple :

$$F(a,b,c) = (a+b+c)(a+\bar{b}+c)(a+b+\bar{c})$$ ⟵ Exemple de maxterme

3-2 Représentation par utilisation d'une table de vérité

La table de vérité d'une fonction logique représente toutes les combinaisons possibles de ses variables logiques. Si le nombre de variables est n, on aura donc 2^n combinaisons possibles.

Exemple :

Soit la fonction **F(a,b,c) = 1** si le nombre des variables à l'état 1 est paire.

La fonction F a trois variables, donc $\mathbf{2^3 = 8}$ combinaisons possibles

Combinaison	a	b	c	F
0	0	0	0	**0**
1	0	0	1	**0**
2	0	1	0	**0**
3	0	1	1	**1**
4	1	0	0	**0**
5	1	0	1	**1**
6	1	1	0	**1**
7	1	1	1	**0**

Equivalence entre table de vérité et formes canoniques :

❖ **Forme canonique SDP:**

Pour former une somme canonique de produits par des mintermes, on utilise les termes où la fonction logique vaut « 1 » dans la table de vérité.

Pour exprimer un minterme:

- On utilise le complément de la variable aux termes ou on a un **« 0 »** dans la table de vérité.
- Si la variable vaut **« 1 »,** on l'écrit sans le complémenter.

❖ **Forme canonique PDS:**

Pour former un produit canonique de sommes par des maxtermes, on utilise les termes où la fonction logique vaut « 0 »

Pour exprimer un maxterme:

- On utilise le complément de la variable aux termes ou on a un **« 1 »** dans la table de vérité.
- Si la variable vaut **« 0 »,** on l'écrit sans le complémenter

Exemple :

Soit la fonction F de l'exemple précédent :

Combinaison	a	b	c	F
0	0	0	0	**0**
1	0	0	1	**0**
2	0	1	0	**0**
3	0	1	1	**1**
4	1	0	0	**0**
5	1	0	1	**1**
6	1	1	0	**1**
7	1	1	1	**0**

Forme SDP : $F(a,b,c) = \bar{a}bc + a\bar{b}c + ab\bar{c}$

Forme PDS : $F(a,b,c) = (a+b+c)(a+b+\bar{c})(a+\bar{b}+c)(\bar{a}+b+c)(\bar{a}+\bar{b}+\bar{c})$

Forme décimale (numérique) SDP: $F(a,b,c) = \sum(3,5,6)$

Forme décimale (numérique) PDS: $F(a,b,c) = \prod(0,1,2,4,7)$

3-3 Représentation par utilisation du tableau de Karnaugh

Le tableau de Karnaugh est une représentation graphique d'une fonction logique. Pour construire le tableau de Karnaugh, on suit les étapes suivantes :

- Pour n variables logiques, le tableau contient 2^n cases.
- Chaque case correspond à une combinaison des variables logiques d'entrée et elle représente la valeur binaire de la fonction logique pour cette combinaison.

- Pour représenter les combinaisons dans ce tableau, on utilise le code binaire réfléchi (code de Gray) afin d'éviter le changement de plusieurs variables en passant d'une case à une autre adjacente.

Exemples :

c \ ab	00	01	11	10
0	1	1	0	1
1	1	0	0	0

cd \ ab	00	01	11	10
00	1	0	0	1
01	0	1	1	1
11	1	1	1	1
10	1	0	0	1

Exemple : Convertir une table de vérité en un tableau de Karnaugh

a	b	c	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0



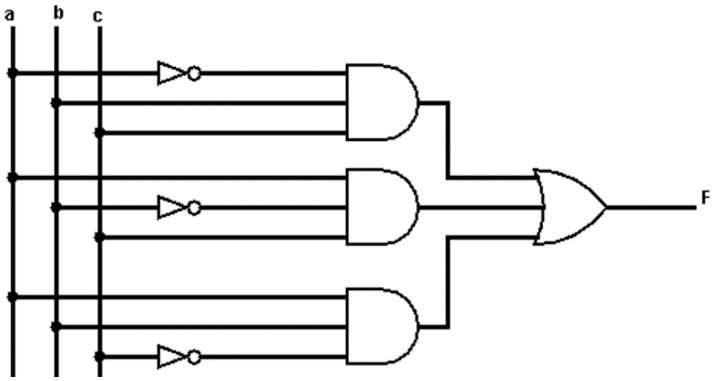
c \ ab	00	01	11	10
0	0	0	1	0
1	0	1	0	1

3-4 Représentation par utilisation d'un logigramme

Un logigramme est la représentation d'une fonction logique à l'aide des symboles des différents opérateurs logiques.

Exemple :

$$F(a,b,c) = \bar{a}bc + a\bar{b}c + ab\bar{c}$$



Représentation des fonctions logiques à l'aide des portes universelles

Chaque fonction logique peut être réalisée par utilisation des portes NAND ou des portes NOR uniquement. Ces deux portes universelles permettent de réaliser toutes les autres portes logiques de bases (OU, ET et NON).

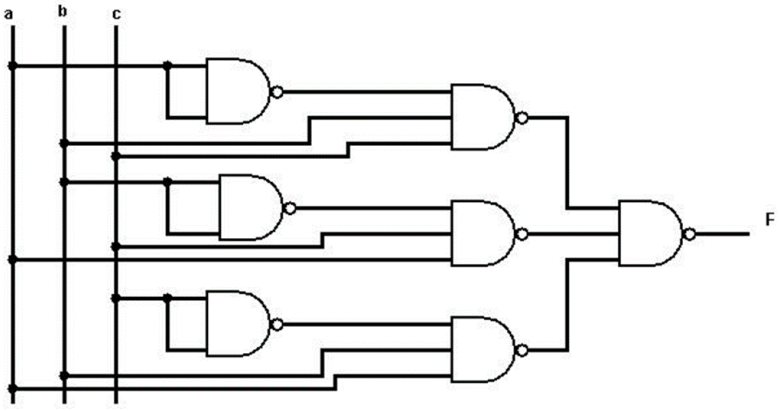
Exemple 1: Réaliser la fonction logique F à l'aide des portes NAND uniquement

$$F(a,b,c) = \bar{a}bc + a\bar{b}c + ab\bar{c}$$

Etapes de réalisation

- Partir d'une fonction logique exprimée sous forme disjonctive (Somme de produits)
- Utiliser le théorème d'involution : $F = \overline{\overline{F}}$
- Développer la barre inférieure à l'aide du théorème de De Morgan
- Utiliser le théorème d'idempotence pour éliminer les inverseurs

$$F = \overline{\overline{F}} = \overline{\overline{\bar{a}bc + a\bar{b}c + ab\bar{c}}} = \overline{\bar{a}bc}.\overline{a\bar{b}c}.\overline{ab\bar{c}} = \overline{\overline{aa}bc}.\overline{a\overline{bb}c}.\overline{ab\overline{cc}}$$

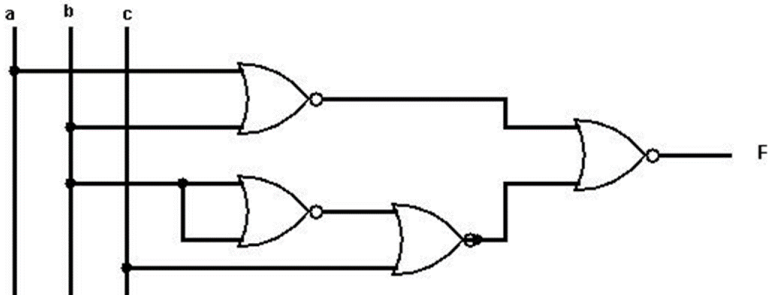


Exemple 2: Réaliser la fonction logique F à l'aide des portes NOR uniquement

$$F(a,b,c) = (a+b)(\bar{b}+c)$$

Mêmes étapes que la porte NAND, mais il faut partir d'une fonction logique exprimée sous forme conjonctive (Produit de sommes)

$$F = \bar{\bar{F}} = \overline{\overline{(a+b)(\bar{b}+c)}} = \overline{\overline{(a+b)} + \overline{(\bar{b}+c)}} = \overline{\overline{(a+b)} + \overline{(\bar{b}+b+c)}}$$



4- Simplification des fonctions logiques

4-1 Simplification algébrique

Pour simplifier algébriquement une fonction logique, on utilise les différentes relations algébriques citées précédemment. La simplification d'une fonction logique permet de la réécrire en utilisant un nombre minimal de termes.

Exemple :

Simplifier algébriquement la fonction F suivante:

$$F = \left(a.\bar{b}.(c + b.d) + \bar{a}.\bar{b}\right).c \quad \Leftarrow$$ 10 opérateurs logiques

$$F = \left(a.\bar{b}.c + a.\bar{b}.b.d + \bar{a}.\bar{b}\right).c$$

Sachant que : $\bar{b}.b = 0; c.c = c; a + \bar{a} = 1$

$$F = \left(a.\bar{b}.c + \bar{a}.\bar{b}\right).c = a.\bar{b}.c.c + \bar{a}.\bar{b}.c = a.\bar{b}.c + \bar{a}.\bar{b}.c = \bar{b}.c.\left(a + \bar{a}\right)$$

$\Rightarrow \quad F = \bar{b}.c \quad \Leftarrow$ 2 opérateurs logiques

4-2 Simplification graphique par utilisation du tableau de Karnaugh

Pour simplifier graphiquement une fonction logique, on utilise les propriétés du tableau de Karnaugh.

❖ **Groupement des cases adjacentes:**

- Un groupement est formé par des cases adjacentes qui contiennent 1.
- Le nombre de cases adjacentes dans un groupement est une puissance de 2.
- Chaque groupement doit contenir un maximum de 1 afin de minimiser le nombre des termes.
- La valeur 1 peut être utilisée dans plusieurs groupements.
- Si une fonction logique est exprimée avec **n variables**, un groupement de **2^k cases adjacentes** conduit à **un produit de (n-k) termes**, les variables éliminées sont celles qui changent de bit.
- La fonction simplifiée par utilisation du tableau de Karnaugh est la réunion de tous les groupements.

Exemples :

a \ b	0	1
0	1	0
1	0	1

$F = \bar{a}\bar{b} + ab$

a \ b	0	1
0	1	0
1	1	1

$F = a + \bar{b}$

a \	00	01	11	10
0	1	0	0	1
1	1	1	1	0

$F = ac + \bar{b}\bar{c} + \bar{a}\bar{c}$

ab \ cd	00	01	11	10
00	0	1	0	0
01	0	1	0	0
11	1	1	0	1
10	1	1	0	1

$F = \bar{a}b + \bar{b}c$

ab \ cd	00	01	11	10
00	0	0	1	1
01	1	1	1	1
11	1	1	1	1
10	0	0	1	1

$F = a + d$

ab \ cd	00	01	11	10
00	1	1	0	1
01	0	1	0	0
11	0	0	0	0
10	1	0	0	1

$F = \bar{b}\bar{d} + \bar{a}b\bar{c}$

ab \ cd	00	01	11	10
00	0	0	1	1
01	0	0	0	0
11	1	0	0	1
10	1	0	1	1

$F = a\bar{d} + \bar{b}c$

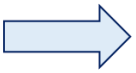
➢ **Valeurs indéterminées ou indéfinies**:

Pour certaines combinaisons binaires, la valeur logique d'une fonction logique n'est pas définie, on dit que la fonction est incomplètement définie, dans ce cas on remplace sa valeur logique par le symbole X ou ϕ. La valeur indéfinie X peut prendre 0 ou bien 1.

La minimisation des fonctions avec des valeurs indéfinies par la méthode de Karnaugh se fait de la même manière en remplaçant les valeurs X par 1 afin de former des groupements avec le maximum des 1.

Exemples :

a	b	c	F
0	0	0	X
0	0	1	0
0	1	0	1
0	1	1	X
1	0	0	0
1	0	1	0
1	1	0	X
1	1	1	1



a \ bc	00	01	11	10
0	X	0	X	1
1	0	0	1	X

$F = b$

cd \ ab	00	01	11	10
00	0	0	X	0
01	0	X	1	X
11	X	1	1	1
10	0	X	1	X

$F = ab + cd$

Chapitre III : Circuits logiques combinatoires

1- Logique combinatoire

La logique combinatoire étudie les fonctions logiques dont les valeurs de sortie ne dépendent que des états logiques des entrées.

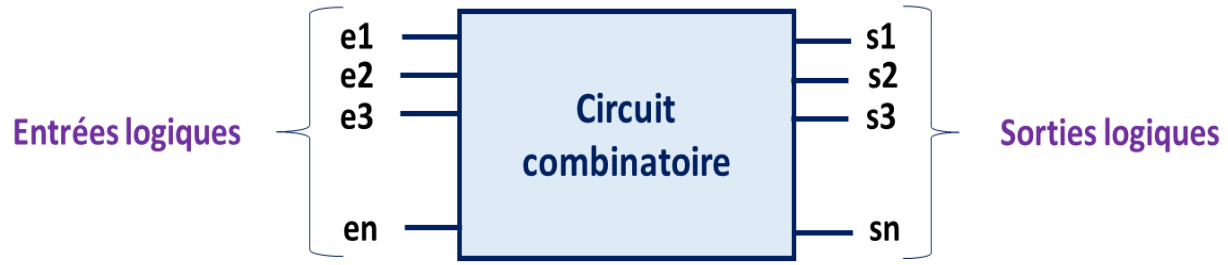


Figure 3.1 : Schéma bloc d'un circuit combinatoire

La réalisation et la conception d'un circuit combinatoire se fait à partir d'une synthèse combinatoire:

- Etablir la table de vérité à partir d'un cahier de charge qui décrit le fonctionnement d'un système.
- Etablir les équations logiques des différentes sorties en fonction des variables d'entrée.
- Simplifier les équations logiques.
- Réalisation du logigramme.

2- Exemples de quelques circuits combinatoires

2-1 Les additionneurs

2-1-1 Demi-additionneur

C'est un circuit combinatoire qui réalise l'addition binaire de deux bits. La sortie de ce circuit donne le résultat de la somme S ainsi que la retenue R.



Figure 3.2 : Schéma bloc d'un demi-additionneur

❖ **Table de vérité :**

a0	b0	S	R
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Tableau 3.1 : Table de vérité d'un demi-additionneur

❖ **Equations logiques et logigramme**

$$\begin{cases} S = a0.\overline{b0} + \overline{a0}.b0 = a0 \oplus b0 \\ R = a0.b0 \end{cases}$$

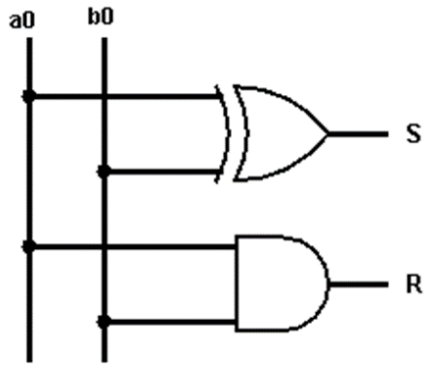


Figure 3.3 : Logigramme d'un demi-additionneur

2-1-2 Additionneur complet

Dans un additionneur complet, on doit prendre en considération la retenue de l'addition précédente.



Figure 3.4 : Schéma bloc d'un additionneur complet

❖ **Table de vérité :**

a_i	b_i	R_{i-1}	S_i	R_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Tableau 3.2 : Table de vérité d'un additionneur complet

❖ **Equations logiques et logigramme**

$$S = \overline{a_i}.\overline{b_i}.R_{i-1} + \overline{a_i}.b_i.\overline{R_{i-1}} + a_i.\overline{b_i}.\overline{R_{i-1}} + a_i.b_i.R_{i-1}$$
$$\rightarrow S = a_i \oplus b_i \oplus R_{i-1}$$

$$R_i = \overline{a_i}.b_i.R_{i-1} + a_i.\overline{b_i}.R_{i-1} + a_i.b_i.\overline{R_{i-1}} + a_i.b_i.R_{i-1}$$
$$\rightarrow R_i = (a_i \oplus b_i)R_{i-1} + a_i.b_i$$

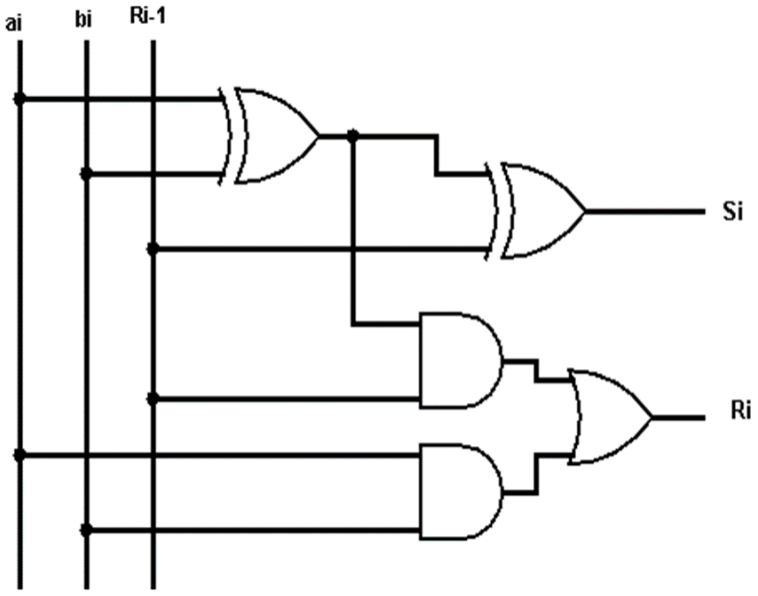


Figure 3.5 : Logigramme d'un additionneur complet

Réalisation d'un additionneur complet par utilisation de deux demi-additionneur :

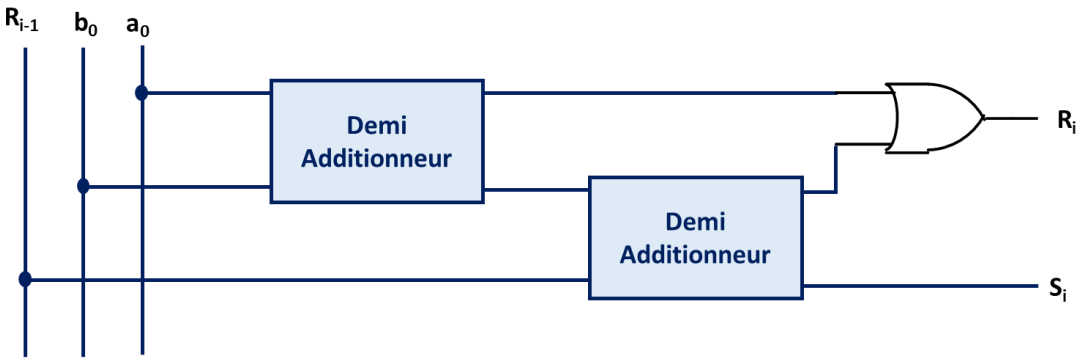


Figure 3.6 : Circuit de réalisation d'un additionneur complet par deux demi-additionneur

Additionneur quatre bits :

On peut réaliser un additionneur à quatre bits en connectant en cascade quatre additionneur complet à deux bits. La retenue initiale du premier additionneur et mise à 0 ($R_0 = 0$).

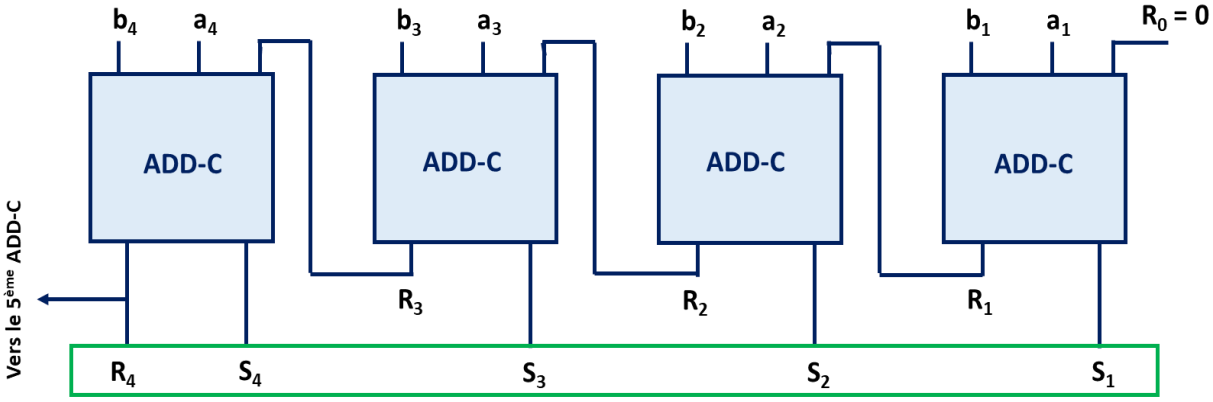


Figure 3.7 : Schéma d'un additionneur à quatre bits

2-2 Les soustracteurs

2-2-1 Demi-soustracteur

C'est un soustracteur de deux bits. La sortie de ce circuit donne le résultat de la soustraction D et la retenue.



Figure 3.8 : Schéma bloc d'un demi-soustracteur

❖ **Table de vérité :**

a0	b0	D	R
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Tableau 3.3 : Table de vérité d'un demi-soustracteur

❖ **Equations logiques et logigramme**

$$\begin{cases} D = a0.\overline{b0} + \overline{a0}.b0 = a0 \oplus b0 \\ R = \overline{a0}.b0 \end{cases}$$

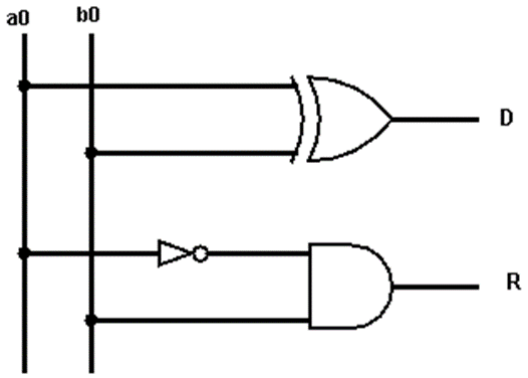


Figure 3.9 : Logigramme d'un demi-soustracteur

2-2-2 Soustracteur complet

Dans ce circuit, on doit prendre en considération la retenue de la soustraction précédente.



Figure 3.9 : Schéma bloc d'un soustracteur complet

❖ **<u>Table de vérité :</u>**

a_i	b_i	R_{i-1}	D_i	R_i
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Tableau 3.4 : Table de vérité d'un soustracteur complet

❖ **Equations logiques et logigramme**

$$D_i = \overline{a_i}.\overline{b_i}.R_{i-1} + \overline{a_i}.b_i.\overline{R_{i-1}} + a_i.\overline{b_i}.\overline{R_{i-1}} + a_i.b_i.R_{i-1}$$
$$\rightarrow D_i = a_i \oplus b_i \oplus R_{i-1}$$

$$R_i = \overline{a_i}.\overline{b_i}.R_{i-1} + \overline{a_i}.b_i.\overline{R_{i-1}} + \overline{a_i}.b_i.R_{i-1} + a_i.b_i.R_{i-1}$$
$$\rightarrow R_i = \overline{(a_i \oplus b_i)}R_{i-1} + \overline{a_i}.b_i$$

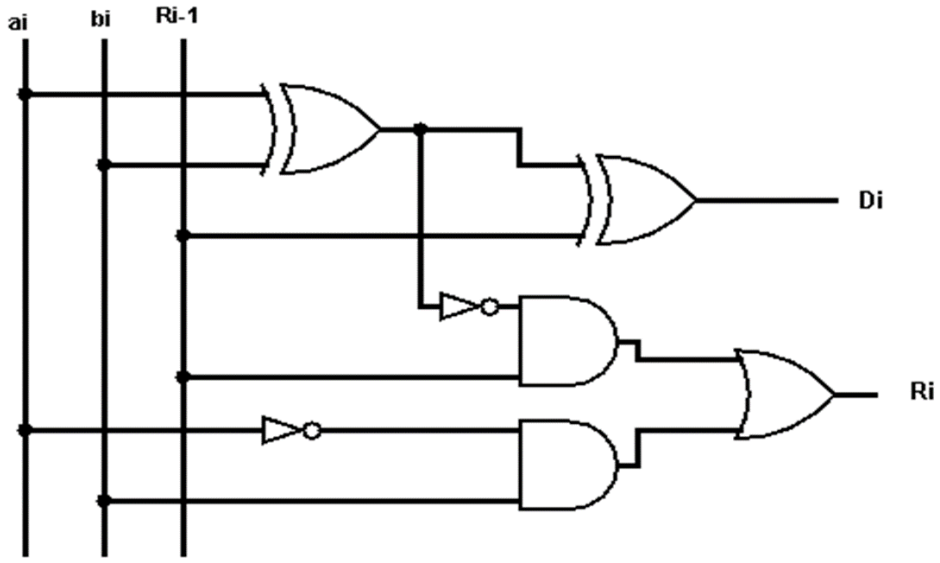


Figure 3.10 : Logigramme d'un soustracteur complet

Réalisation d'un soustracteur complet par deux demi-soustracteurs :

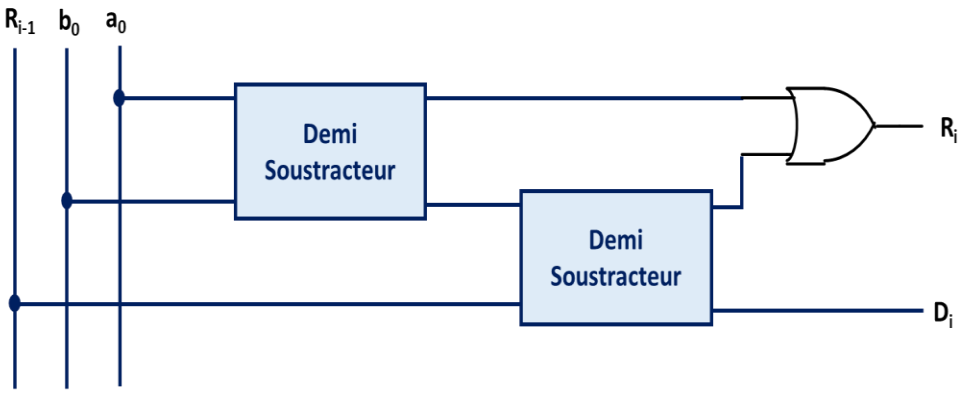


Figure 3.11 : Circuit de réalisation d'un soustracteur complet par deux demi-soustracteurs

2-3 Les comparateurs

C'est un circuit combinatoire qui permet de comparer deux nombres binaire A et B en entrée. Ce circuit possède trois sorties: la sortie E (Egalité) si A = B, la sortie S (Supérieur) si A > B et la sortie I (Inférieur) si A < B.



Figure 3.12 : Schéma bloc d'un comparateur

❖ **Table de vérité :**

a_0	b_0	S	E	I
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

Tableau 3.5 : Table de vérité d'un comparateur

❖ **Equations logiques et logigramme**

$$\begin{cases} S = a_0\overline{b_0} \\ E = \overline{a_0 b_0} + a_0 b_0 = \overline{a_0 \oplus b_0} \\ I = \overline{a_0} b_0 \end{cases}$$

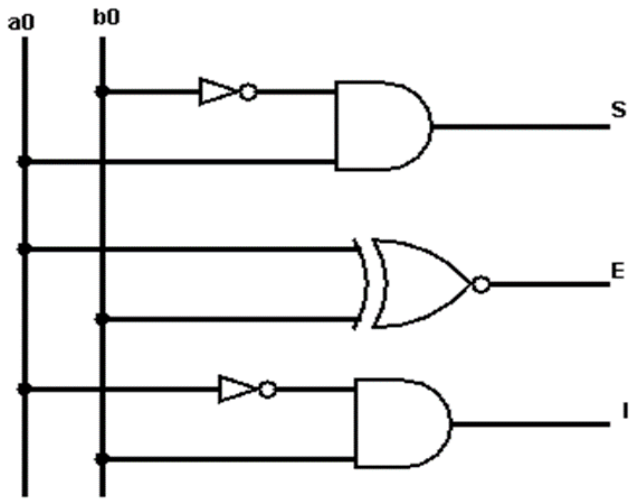


Figure 3.13 : Logigramme d'un comparateur

2-4 Les codeurs/décodeurs

- **Codeur**

C'est un circuit combinatoire qui possède $N = 2^n$ entrées dont une seule est active à la fois, et fournie en sortie un code binaire de n bits correspondant à chaque entrée.

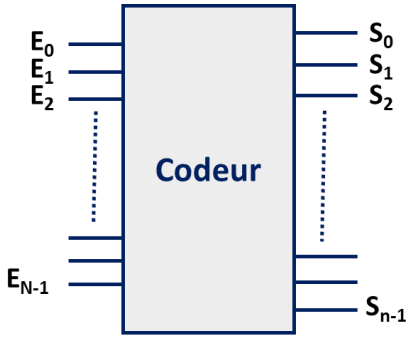


Figure 3.14 : Schéma bloc d'un codeur

Exemple d'un codeur de 4 lignes vers 2 :



Figure 3.15 : Schéma bloc d'un codeur 4 lignes vers 2

- **Table de vérité :**

E_3	E_2	E_1	E_0	S_1	S_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Table 3.6 : Table de vérité d'un codeur 4 lignes vers 2

❖ **Equations logiques et logigramme**

$$S_0 = E_1 + E_3$$
$$S_1 = E_2 + E_3$$

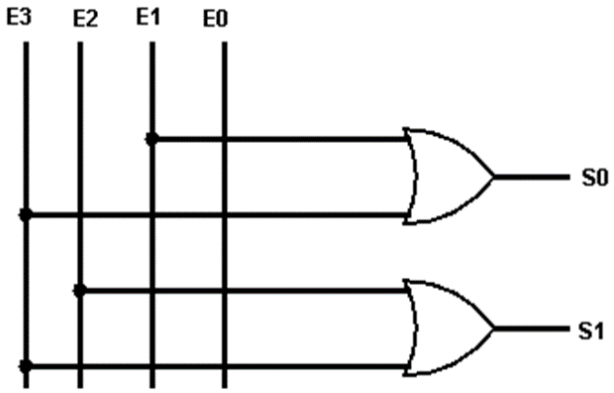


Figure 3.16: logigramme d'un codeur 4 lignes vers 2

➢ **Décodeur**

C'est un circuit combinatoire qui possède une entrée binaire de n bits permettant 2^n combinaisons possibles, et N sorties (2^n >= N).

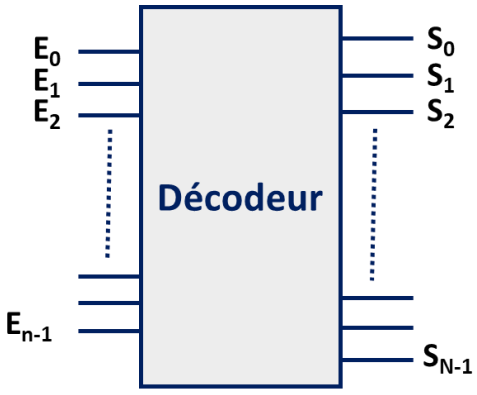


Figure 3.17: Schéma bloc d'un décodeur

Exemple d'un décodeur 1 parmi 4 (2 lignes vers 4):



Figure 3.18: Schéma bloc d'un décodeur 1 parmi 4

❖ **Table de vérité :**

E_1	E_0	S_0	S_1	S_2	S_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Table 3.7 : Table de vérité d'un décodeur 2 lignes vers 4

❖ **Equations logiques et logigramme**

$S_0 = \overline{E_1}\,\overline{E_0}$

$S_1 = \overline{E_1}E_0$

$S_2 = E_1\overline{E_0}$

$S_3 = E_1E_0$

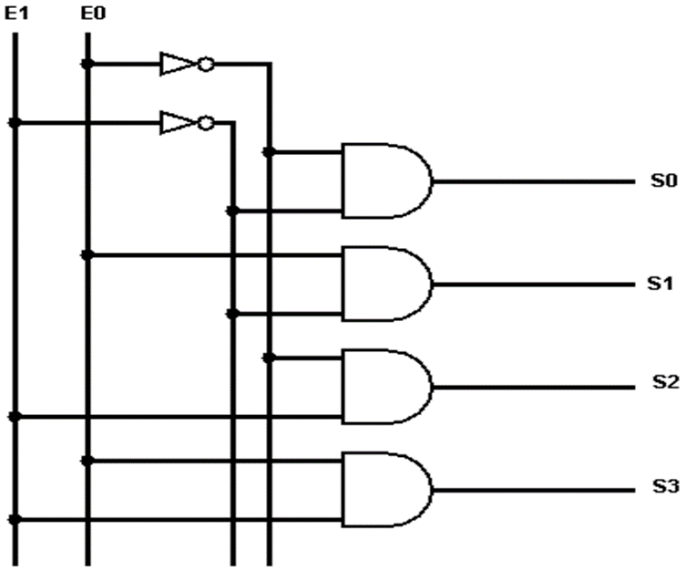


Figure 3.19: logigramme d'un décodeur 2 lignes vers 4

Exemple d'un décodeur BCD sept segments :

- **Afficheur 7 segments**

Un afficheur sept segment est un système d'affichage à sept segments (sept diodes LEDs) qui permet d'afficher des chiffres décimales ou hexadécimales. L'allumage des diodes est commandé selon le chiffre souhaité.

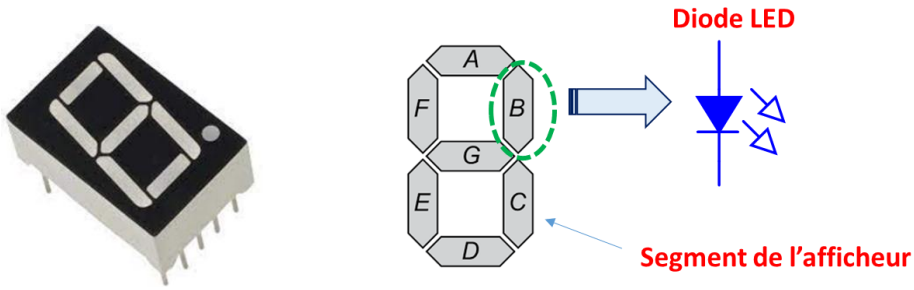


Figure 3.20: Afficheur 7 segments

Il existe deux types d'afficheurs sept segments, un afficheur à cathode commune et un autre à anode commune.

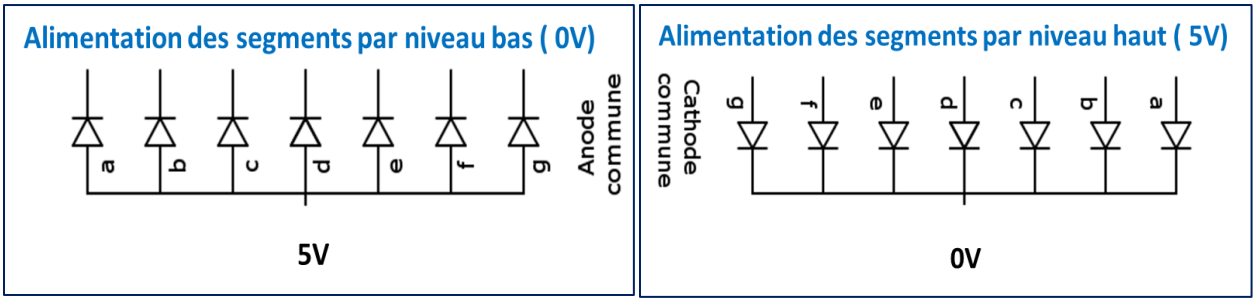


Figure 3.21: Afficheur 7 segments à anode commune (à gauche) et à anode commune (à droite)

- **Décodeur BCD 7 segments :**

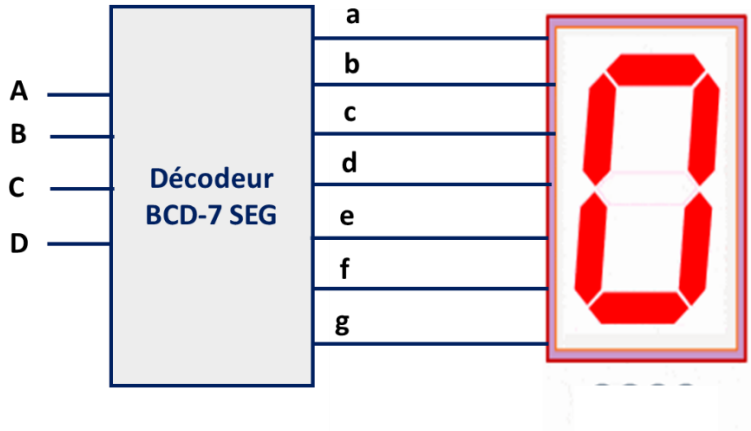


Figure 3.21: Décodeur BCD 7 segments

Table de vérité :

Décimal	DCBA	a	b	c	d	e	f	g
0	0000	1	1	1	1	1	1	0
1	0001	0	1	1	0	0	0	0
2	0010	1	1	0	1	1	0	1
3	0011	1	1	1	1	0	0	1
4	0100	0	1	1	0	0	1	1
5	0101	1	0	1	1	0	1	1
6	0110	1	0	1	1	1	1	1
7	0111	1	1	1	0	0	0	0
8	1000	1	1	1	1	1	1	1
9	1001	1	1	1	1	0	1	1

Table 3.8 : Table de vérité d'un décodeur BCD 7 segments

2-5 Transcodeurs

C'est un circuit combinatoire qui permet de convertir une information écrite selon un **code 1** vers une autre information selon un **code 2**.

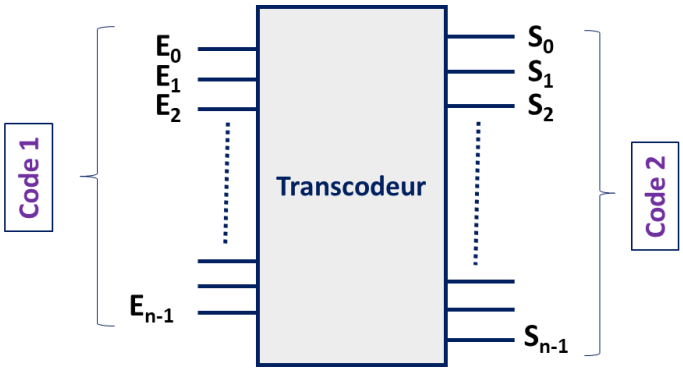


Figure 3.22 : Schéma bloc d'un transcodeur

Exemple: Transcodeur binaire naturel en binaire réfléchi (3 bits)



Figure 3.23 : Schéma bloc d'un transcodeur 3 bits

Table de vérité :

a	b	c	ar	br	cr
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	1	1	0
1	0	1	1	1	1
1	1	0	1	0	1
1	1	1	1	0	0

Table 3.9 : Table de vérité d'un transcodeur 3 bits

Equations logiques :

	00	01	11	10
0	0	0	0	0
1	1	1	1	1

$ar = a$

	00	01	11	10
0	0	0	1	1
1	1	1	0	0

$br = a\bar{b} + \bar{a}b = a \oplus b$

	00	01	11	10
0	0	1	0	1
1	0	1	0	1

$cr = b\bar{c} + \bar{b}c = b \oplus c$

Logigramme :

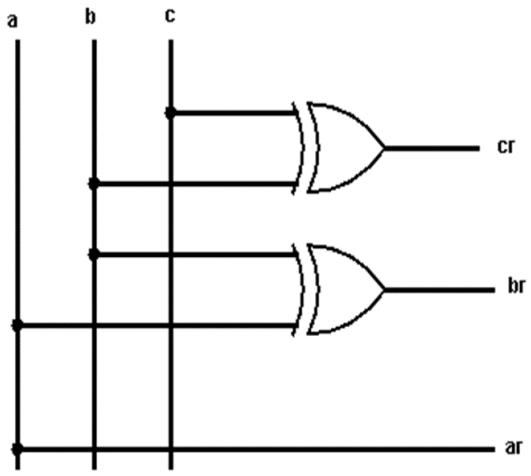


Figure 3.24 : Logigramme d'un transcodeur 3 bits

2-6 Multiplexeurs/Démultiplexeurs

➢ **Multiplexeur**

C'est un circuit combinatoire qui possède 2^n entrées, n entrées de sélection et une seule sortie. On le note MUX 2^n vers 1.

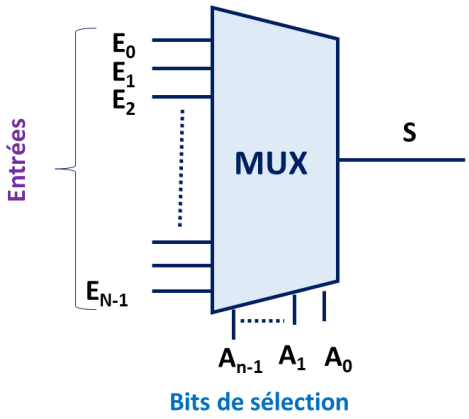


Figure 3.25 : Schéma bloc d'un multiplexeur

Exemple: Multiplexeur 4 vers 1 :

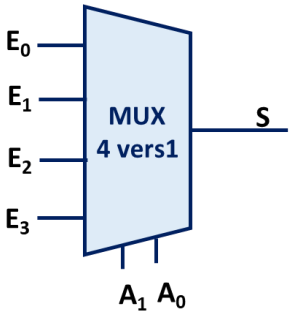


Figure 3.26 : Schéma bloc d'un multiplexeur 4 vers 1

Table de vérité :

A_1	A_0	S
0	0	E_0
0	1	E_1
1	0	E_2
1	1	E_3

Table 3.10 : Table de vérité d'un multiplexeur 4 vers 1

Equation logique :

$$S = \overline{A_1}\,\overline{A_0}E_0 + \overline{A_1}A_0E_1 + A_1\overline{A_0}E_2 + A_1A_0E_3$$

Logigramme :

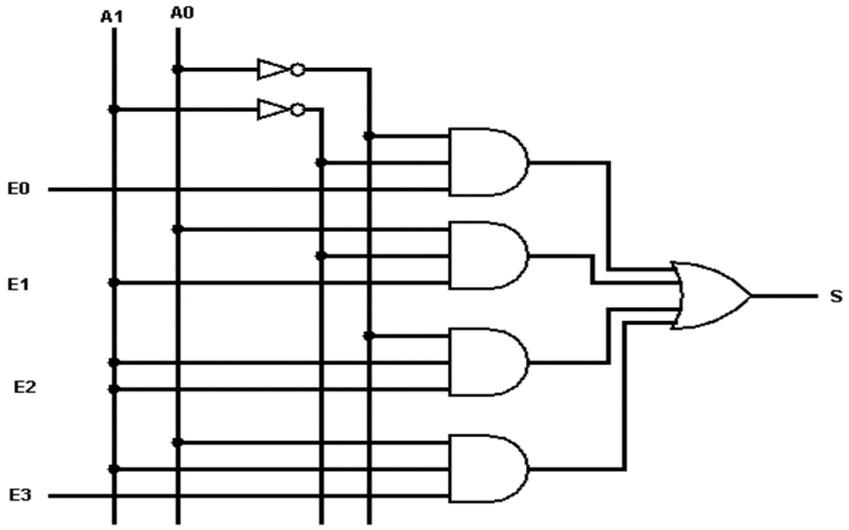


Figure 3.27 : Logigramme d'un multiplexeur 4 vers 1

- **Démultiplexeur**

C'est un circuit combinatoire qui possède une seule entrée, n entrées de sélection et 2^n sorties.

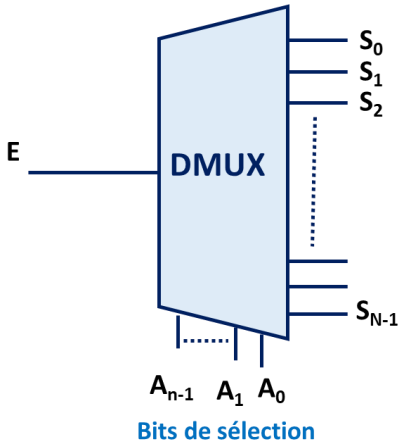


Figure 3.28 : Schéma bloc d'un démultiplexeur

Exemple: Multiplexeur 4 vers 1 :

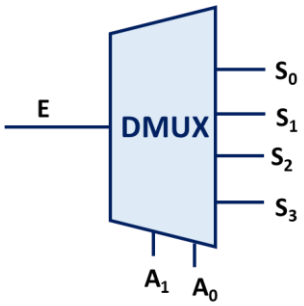


Figure 3.29 : Schéma bloc d'un démultiplexeur 1 vers 4

Table de vérité :

A_1	A_0	S_0	S_1	S_2	S_3
0	0	E	0	0	0
0	1	0	E	0	0
1	0	0	0	E	0
1	1	0	0	0	E

Table 3.11 : Table de vérité d'un démultiplexeur 1 vers 4

Equations logiques :

$$S_0 = \overline{A_1}\,\overline{A_0}E$$
$$S_1 = \overline{A_1}A_0E$$
$$S_2 = A_1\overline{A_0}E$$
$$S_3 = A_1A_0E$$

Logigramme :

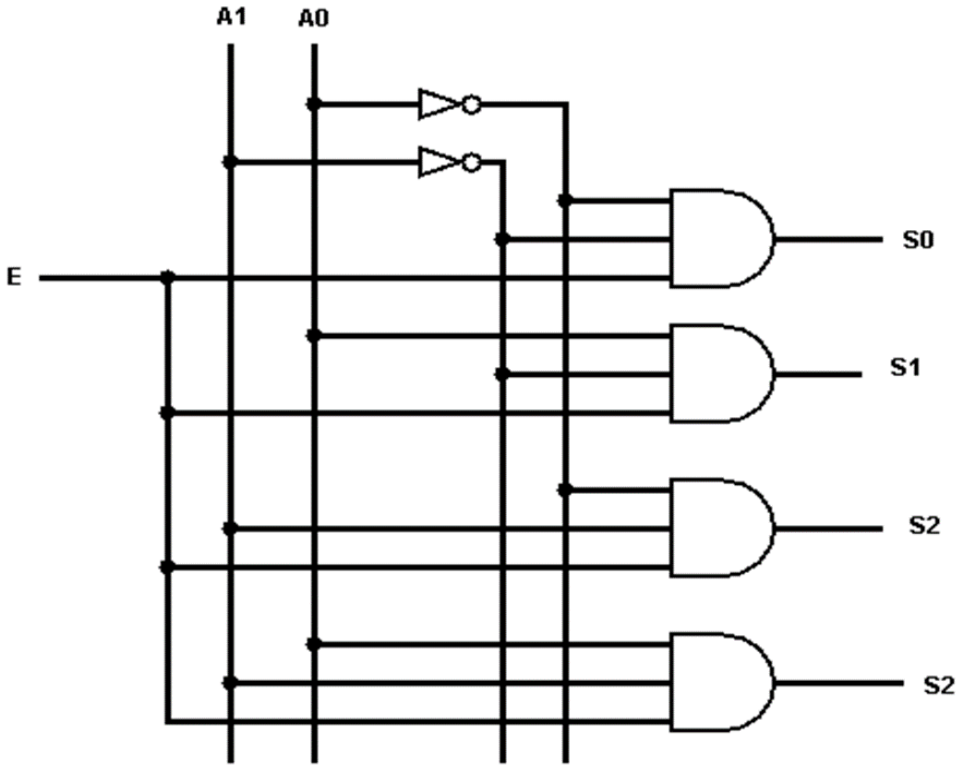


Figure 3.30: Logigramme d'un démultiplexeur 1 vers 4

- **Applications des multiplexeurs:**

Parmi les applications des multiplexeurs, on trouve la réalisation des fonctions logique. Cela permet de réduire le nombre de circuits intégrés utilisées pour la réalisation basée sur des portes

logiques de base. L'exemple ci-dessous montre bien la réalisation d'une fonction logique à partir de sa table de vérité.

a	b	c	F	S
0	0	0	0	E_0
0	0	1	0	E_1
0	1	0	0	E_2
0	1	1	1	E_3
1	0	0	0	E_4
1	0	1	1	E_5
1	1	0	1	E_6
1	1	1	0	E_7

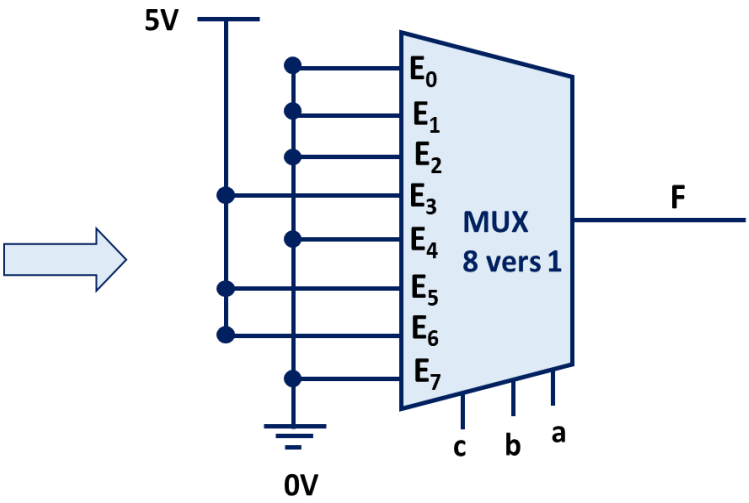


Figure 3.31: Exemple de réalisation d'une fonction logique par utilisation d'un multiplexeur

Chapitre IV : Logique séquentielle et bascules

1- Système séquentiel

Un système basé sur la logique séquentielle est un système logique dont les variables de sorties dépendent des états d'entrées et aussi des états précédents des variables de sorties. En outre, un système séquentiel mémorise l'état précédent de ses sorties.

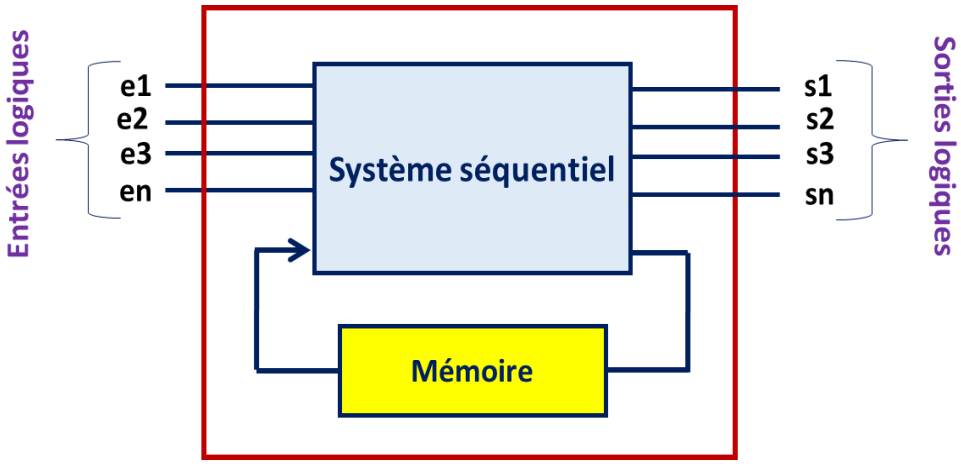


Figure 4.1 : Schéma bloc d'un système séquentiel

2- Bascules

2-1 Définitions

Une bascule est l'élément de base de la logique séquentielle, elle a deux états stables, on l'appelle aussi un bistable. En plus, une bascule est une mémoire élémentaire qui permet de mémoriser une information binaire à un bit.

Selon le fonctionnement des bascules, on distingue deux types :

- Asynchrone : les sorties de la bascule peuvent changer d'état à tout instant.
- Synchrone: utilisation d'un signal d'horloge pour la synchronisation.

2-2 Bascule RS asynchrone

La bascule RS asynchrone a deux entrées S (Set: Mise à 1) et R (Reset: Mise à 0) et deux sorties complémentaires.

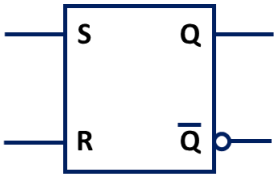


Figure 4.2 : Symbole d'une bascule RS asynchrone

Table de vérité d'une bascule RS :

S	R	Qn	$\overline{Qn}$	Fonctionnement
0	0	Q_{n-1}	Q_{n-1}	Mémorisation
0	1	0	1	Mise à 0 (Reset)
1	0	1	0	Mise à 1 (Set)
1	1	-	-	Etat interdit

Table 4.1 : Table de vérité d'une bascule RS

- **Bascule RS à arrêt prioritaire**

Table de vérité

S	R	Qn-1	Qn
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	**0**
1	1	1	**0**

Table 4.2 : Table de vérité d'une bascule RS à arrêt prioritaire

Equation logique

Qn-1 \ SR	00	01	11	10
0	0	0	0	1
1	1	0	0	1

$$Q_n = \overline{R}.(S + Q_{n-1})$$

Logigramme : Réalisation à base de portes NOR

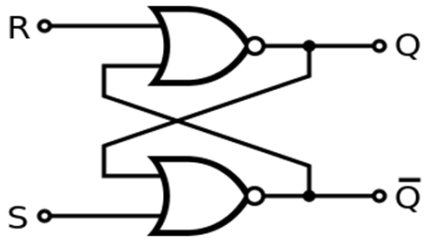


Figure 4.3 : Logigramme d'une bascule RS à arrêt prioritaire par des portes NOR

Remarque : Le cas ou S = R = 1 est caractérisé par un état interdit car : $Q = \overline{Q} = 0$

- **Bascule RS à marche prioritaire**

Table de vérité

S	R	Qn-1	Qn
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Table 4.3 : Table de vérité d'une bascule RS à marche prioritaire

Equation logique

Qn-1 \ SR	00	01	11	10
0	0	0	1	1
1	1	0	1	1

$$Q_n = \overline{R}.(S + Q_{n-1})$$

Logigramme : Réalisation à base de portes NAND

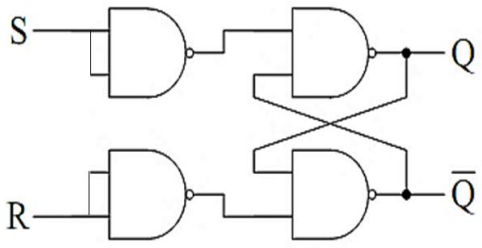


Figure 4.4 : Logigramme d'une bascule RS à marche prioritaire par des portes NAND

Remarque : Le cas ou S = R = 1 est caractérisé par un état interdit car : $Q = \overline{Q} = 1$

Exemple de chronogramme d'une bascule RS asynchrone

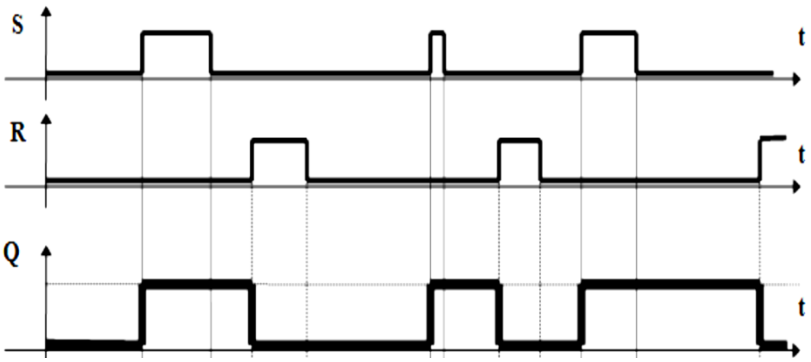


Figure 4.5 : Exemple de chronogramme d'une bascule RS

2-3 Bascules synchrones

Dans une bascule synchrone, le changement d'état de la sortie est autorisé par un signal de commande appelé signal d'horloge. On distingue quatre modes de synchronisation du signal d'horloge : synchronisation sur niveau haut, niveau bas, front montant et front descendant.

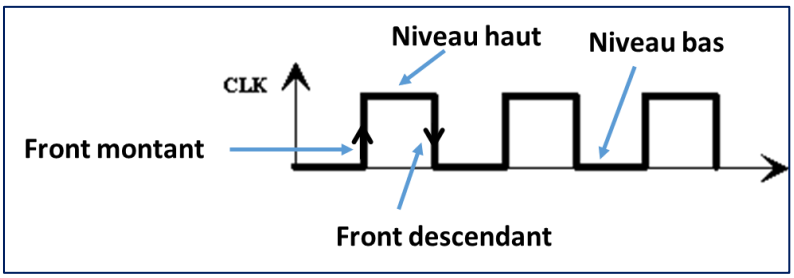


Figure 4.6 : Différents mode de synchronisation des bascules synchrones

2-3-1 Bascule RSH

Les variables de sortie sont commandées par les entrées de mise à zéro R et de mise à un s et aussi par un signal d'horloge H. Selon la synchronisation du signal d'horloge on distingue quatre types de bascules RSH.

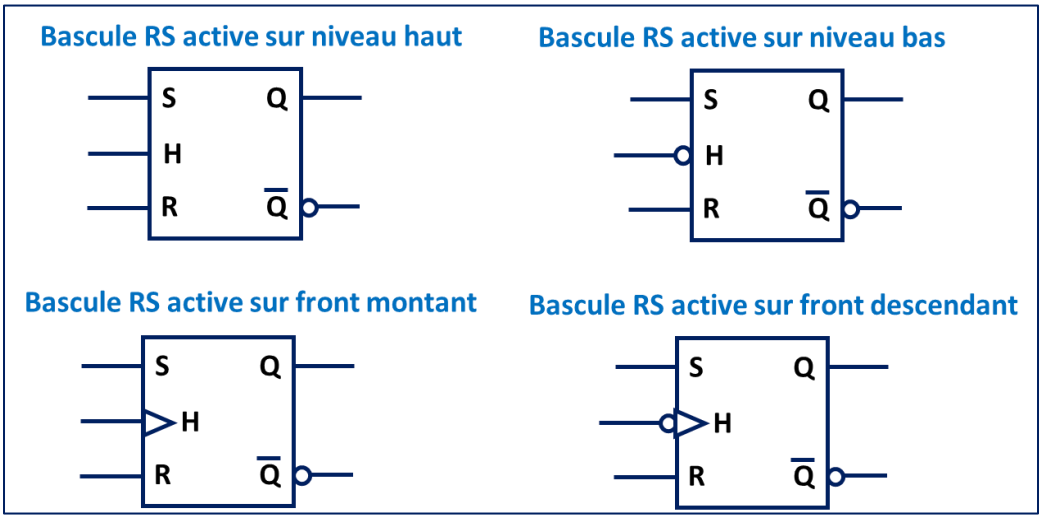


Figure 4.7 : Différents types de bascule synchrone RSH

La bascule RSH est réalisée par utilisation d'une bascule RS et en introduisant le signal d'horloge H.

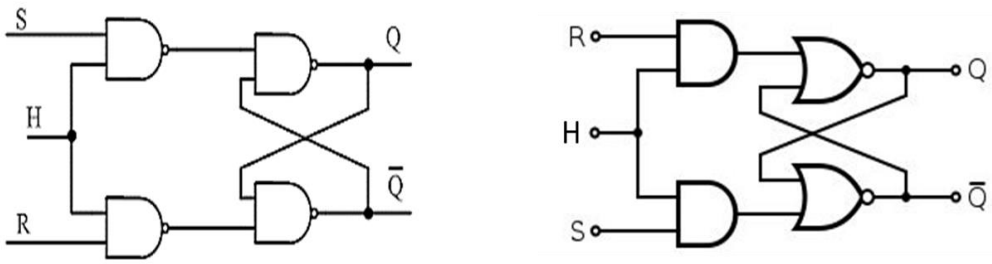


Figure 4.8 : Logigramme d'une bascule RSH

Table de vérité : Bascule RSH à déclenchement sur niveau haut

H	S	R	Qn	Fonctionnement
0	X	X	Q_{n-1}	Mémorisation
1	0	0	Q_{n-1}	Mémorisation
1	0	1	0	Mise à 0 (Reset)
1	1	0	1	Mise à 1 (Set)
1	1	1	-	Etat interdit

Table 4.4 : Table de vérité d'une bascule RSH active sur niveau haut

Exemple de chronogramme d'une bascule RSH synchronisée sur front montant :

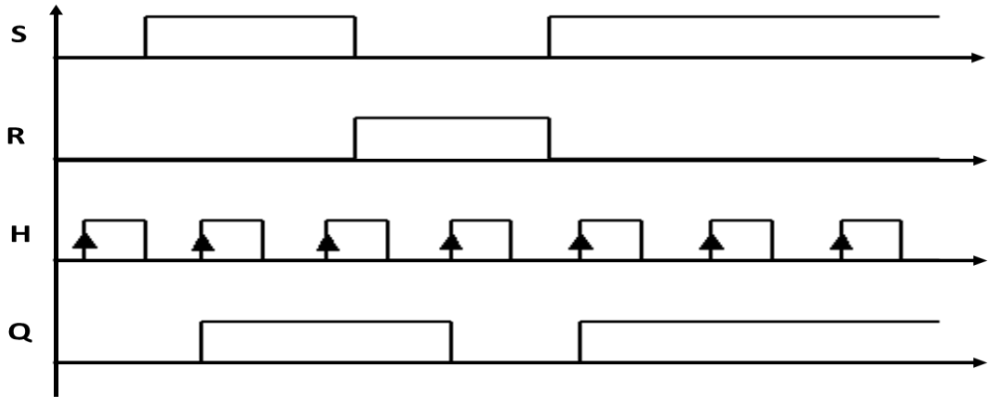


Figure 4.9 : Exemple de chronogramme d'une bascule RSH active sur front montant

2-3-2 Bascule JK

La bascule JK possède deux entrées J (Mise à 1) et K (Mise à 0) et une entrée de synchronisation H (ou CLK). Cette bascule est réalisée en utilisant une bascule RS et en rebouclant les sorties sur les entrées.

Contrairement à la bascule RS, le cas J=K=1 ne donne pas un état interdit mais il donne un basculement: la sortie change d'état à chaque niveau bas ou niveau haut ou front montant ou front descendant de l'horloge.

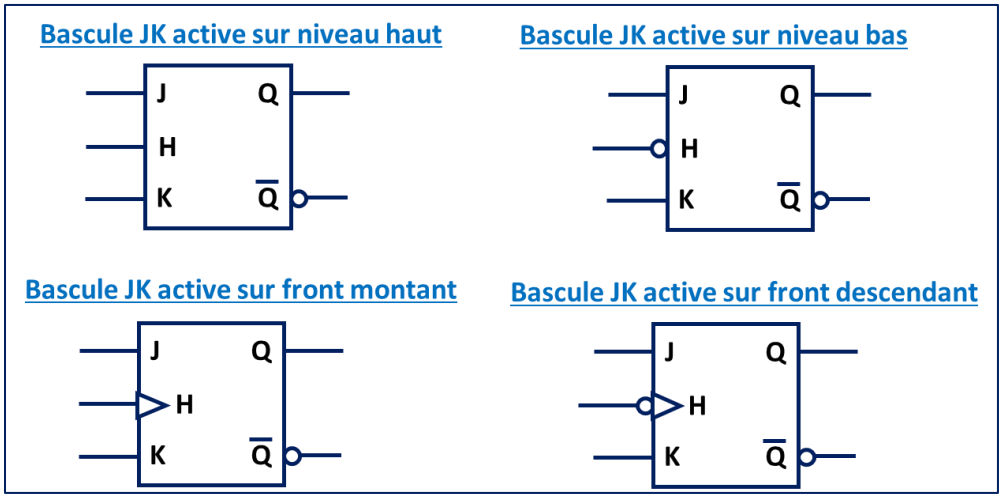


Figure 4.10 : Différents types de bascules JK

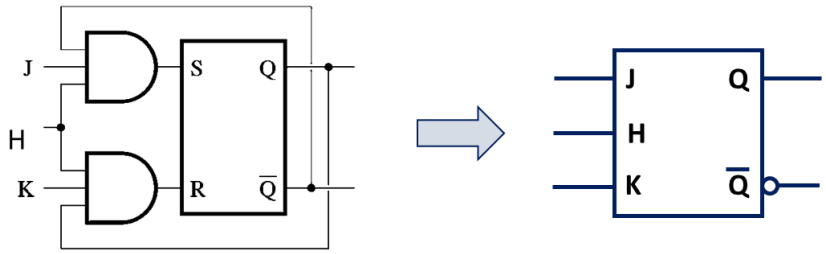


Figure 4.11 : Exemple de réalisation d'une bascule JK active sur niveau haut par utilisation d'une bascule RS

Table de vérité : Bascule JK active sur front montant

H	J	K	Q_n	Fonctionnement
0, 1, ↓	X	X	Q_{n-1}	Mémoire
↑	0	0	Q_{n-1}	Mémoire
↑	0	1	0	Mise à 0
↑	1	0	1	Mise à 1
↑	1	1	$\overline{Q_{n-1}}$	Basculement

Table 4.5 : Table de vérité d'une bascule JK active sur front montant

Equation logique d'une bascule JK

	00	01	11	10
0	0	0	1	1
1	1	0	0	1

$$Q_n = J.\overline{Q_{n-1}} + \overline{K}Q_{n-1}$$

Exemple de chronogramme: Bascule JK à front montant

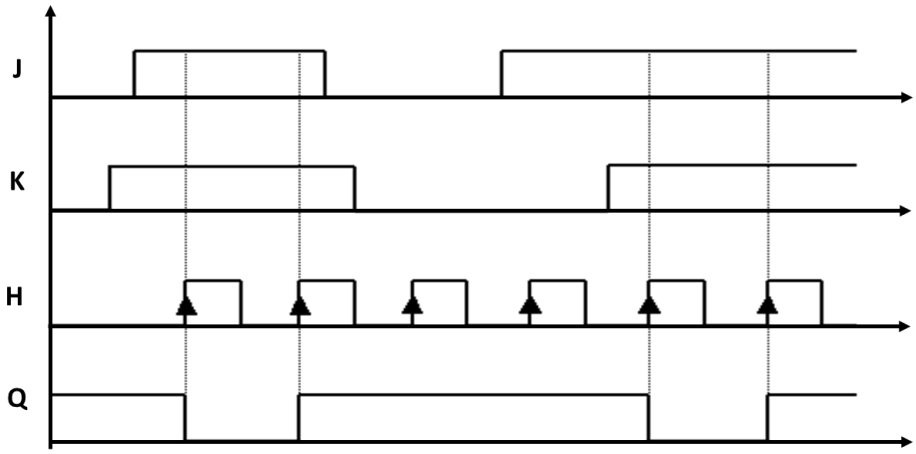


Figure 4.12 : Exemple de chronogramme d'une bascule JK active sur front montant

2-3-3 Bascule D

La bascule D possède une entrée D (Donnée) et une entrée de synchronisation H (ou CLK). Elle est réalisée en utilisant une bascule RS ou une bascule JK en ajoutant un inverseur sur l'entrée R ou K.

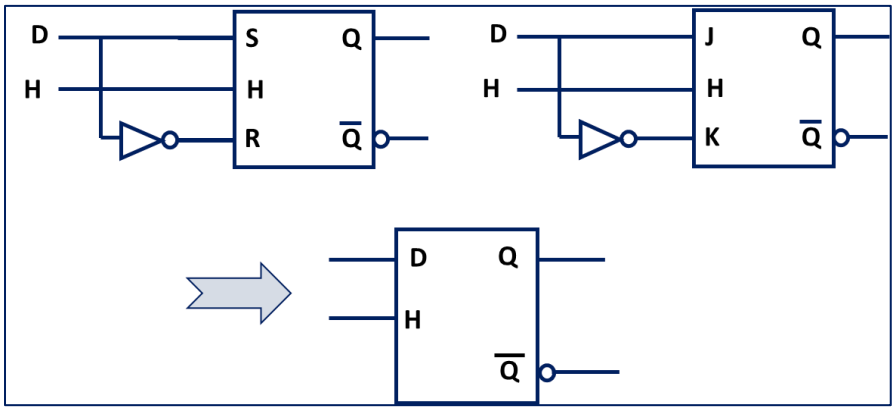


Figure 4.13 : Réalisation d'une bascule D par utilisation d'une bascule JK ou une bascule RSH

On distingue deux types de bascule D:

- Bascule D à verrouillage ou Latch (Data ou Donnée) synchronisée sur niveau haut d'horloge H (ou CLK)

Table de vérité d'une bascule D Latch

H	D	Qn	Fonctionnement
0	X	Q_{n-1}	Mémorisation
1	0	0	Mise à 0
1	1	1	Mise à 1

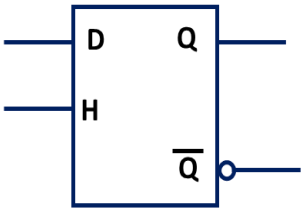


Table 4.6 : Table de vérité d'une bascule D active sur niveau haut

- Bascule D Flip-Flop synchronisée sur front d'horloge H (ou CLK)

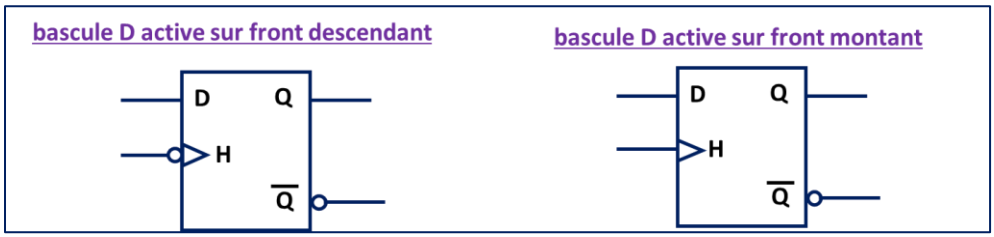


Figure 4.14 : Bascule D active sur front

Table de vérité : Exemple d'une bascule D active sur front montant

H	D	Qn	Fonctionnement
0,1,↓	X	Q_{n-1}	Mémorisation
↑	0	0	Mise à 0
↑	1	1	Mise à 1

Table 4.7 : Table de vérité d'une bascule D active sur front montant

Equation logique :

Dans les deux cas de bascule D latch ou active sur front, l'équation de la sortie est donnée par :

$$Q_n = D$$

Exemple de chronogramme d'une bascule D active sur front montant

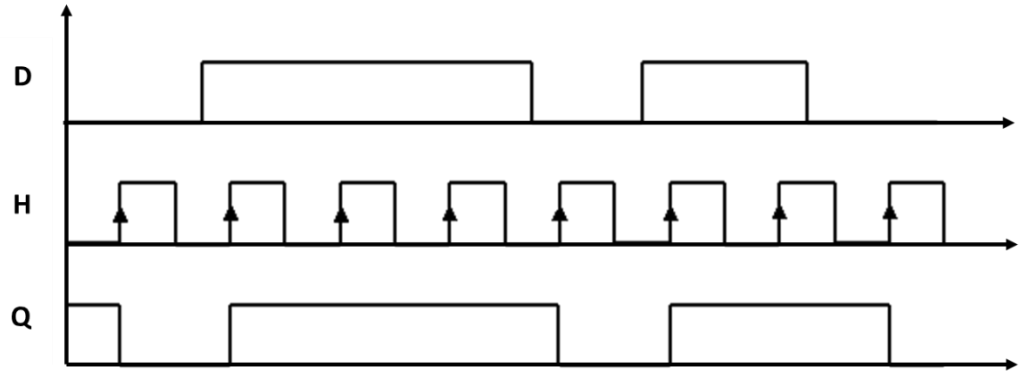


Figure 4.15 : Exemple de chronogramme d'une bascule D active sur front montant

2-3-4 Bascule T

C'est une bascule qui possède une seule entrée T (Toggle) qui constitue aussi l'entrée de l'horloge. La sortie change d'état à chaque front de l'horloge.

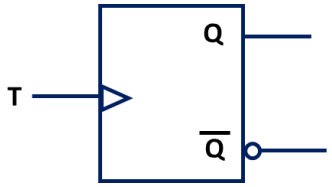


Figure 4.16 : Bascule T

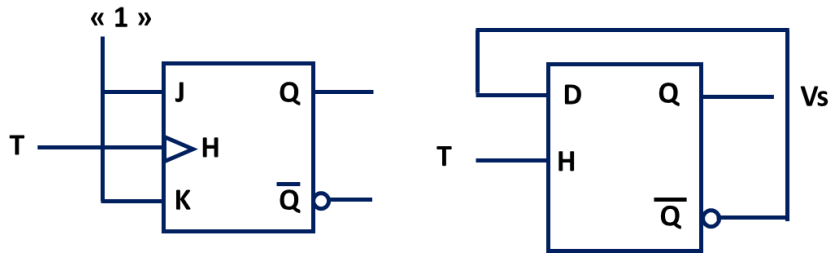


Figure 4.17 : Réalisation d'une bascule T par utilisation d'une bascule JK ou d'une bascule D

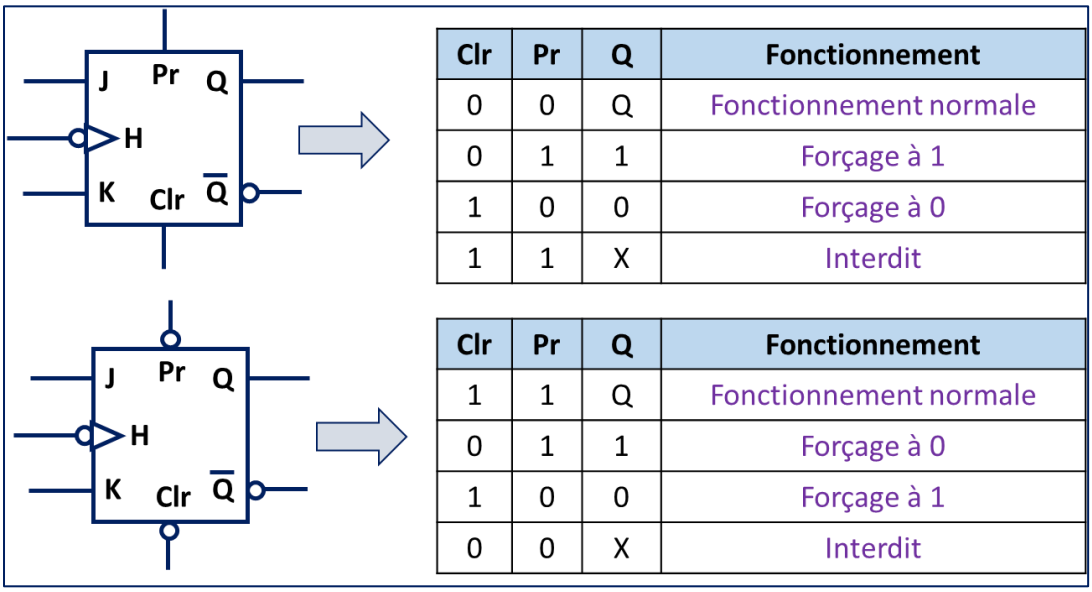
Table de vérité :

H	Qn	Fonctionnement
0	Q_{n-1}	Mémorisation
↑	$\overline{Q_{n-1}}$	Basculement

2-4 Fonctionnement forcé des bascules

Certaines bascules commerciales possèdent des entrées de forçage asynchrones **Clr (Clear)** pour remettre à 0 la sortie ou **Preset (Pr)** pour remettre à 1 la sortie. Les entrées asynchrones peuvent être actives sur niveau bas ou bien sur niveau haut.

Exemple de bascule JK à entrées asynchrones :



Clr	Pr	Q	Fonctionnement
0	0	Q	Fonctionnement normale
0	1	1	Forçage à 1
1	0	0	Forçage à 0
1	1	X	Interdit

Clr	Pr	Q	Fonctionnement
1	1	Q	Fonctionnement normale
0	1	1	Forçage à 0
1	0	0	Forçage à 1
0	0	X	Interdit

Figure 4.18 : Symboles et table de vérité d'une bascule JK à entrées de forçage asynchrone. Les entrées asynchrones sont actives sur niveau haut (bascule en haut) ou bien sur niveau bas (bascule en bas)

Chapitre V : Compteurs et registres

1- Compteurs

1-1 Définitions

Un compteur est un système séquentiel réalisé par un ensemble de bascules connectées entre eux par des portes logiques et synchronisées par un signal d'horloge. L'état d'un compteur est défini par la combinaison des états des différentes bascules.

- **Modulo d'un compteur :**

Un compteur modulo N est un compteur dont le cycle de comptage évolue entre 0 et N-1. Pour faire la synthèse d'un compteur et réaliser le cycle de comptage souhaité, il faut définir les équations de commande des différentes bascules.

- **Types de compteurs :**
  - Compteurs asynchrones : les états des bascules évoluent successivement en cascade.
  - Compteurs synchrones : les états des bascules évoluent simultanément au rythme de la même horloge appliquée à l'entrée de chaque bascule.

1-2 Compteurs/décompteurs asynchrones

1-2-1 Compteurs asynchrones modulo 2^n (cycle complet)

Le signal d'horloge déclenche la première bascule dont la sortie sert d'horloge de la bascule suivante et ainsi de suite jusqu'à la nième bascule. Généralement, on utilise des bascules à déclenchement sur front montant ou descendant. Les bascules dans un compteur asynchrone doivent être commandées en mode de basculement : toutes les bascules doivent avoir J = K = 1.

Un compteur modulo 2^n nécessite n bascules.

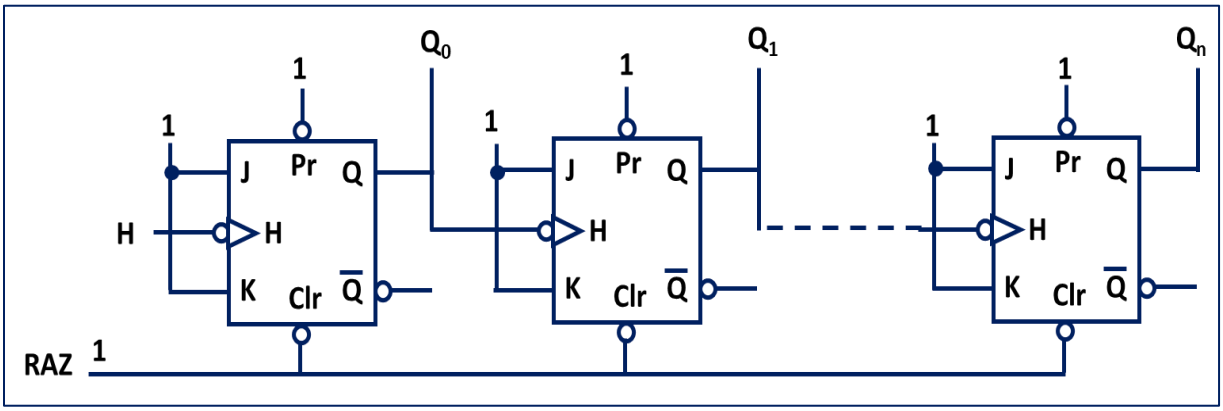


Figure 5.1 : Circuit général d'un compteur asynchrone modulo 2^n par bascules JK actives sur front descendant

Remarques :

- Dans le cas des bascules actives sur front descendant, la sortie Q de chaque bascule devient le signal d'horloge de la bascule suivante.
- Dans le cas des bascules actives sur front montant, la sortie complémentaire de chaque bascule devient le signal d'horloge de la bascule suivante.

Exemple: Compteur asynchrone modulo 8 réalisé par bascules JK

Pour réaliser ce compteur, il faut 3 bascules JK car modulo = 8 = 2^3.

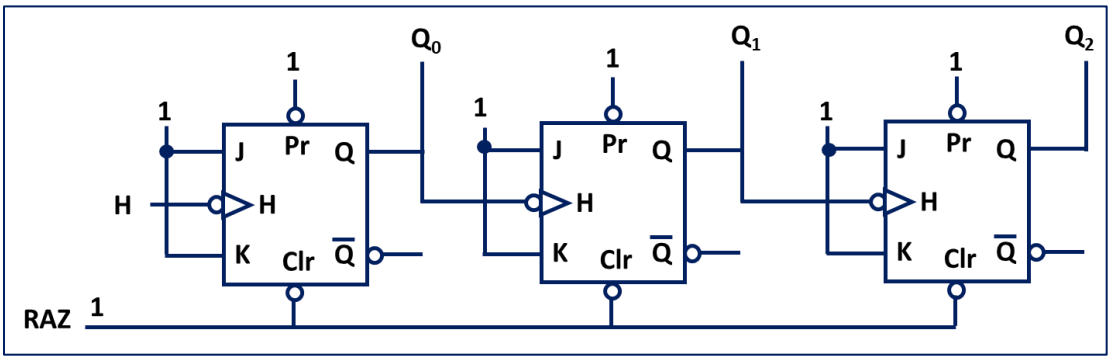


Figure 5.2 : Compteur asynchrone modulo 8

La figure ci-dessous montre le chronogramme de la sortie de chaque bascule du compteur asynchrone modulo8.

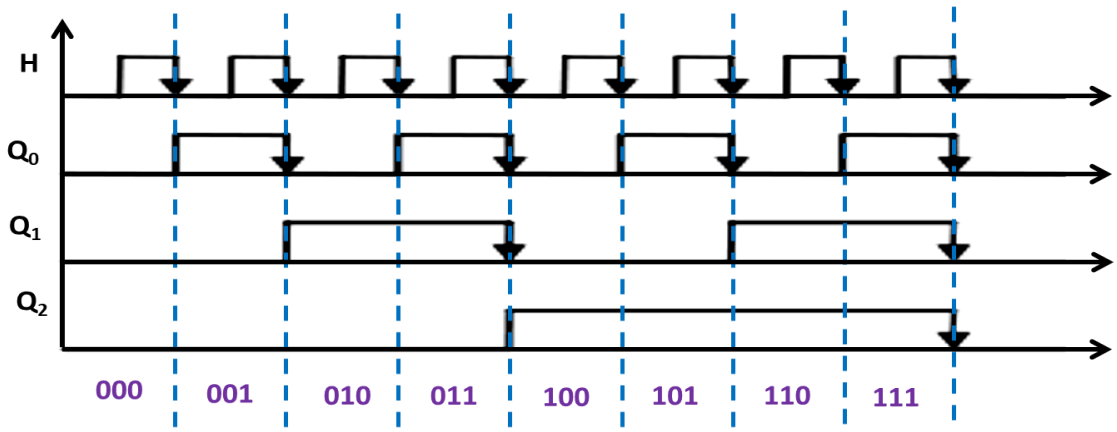


Figure 5.3 : Chronogramme d'un compteur asynchrone modulo 8

1-2-2 Décompteurs asynchrones modulo 2^n (cycle complet)

Le décompteur asynchrone modulo N = 2^n réalise un cycle de comptage de N-1 jusqu'à 0.

Remarques :

- Dans le cas des bascules actives sur front descendant, la sortie complémentaire de chaque bascule devient le signal d'horloge de la bascule suivante.

- Dans le cas des bascules actives sur front montant, la sortie Q de chaque bascule devient le signal d'horloge de la bascule suivante.

Exemple: Décompteur asynchrone modulo 8 réalisé par bascules JK

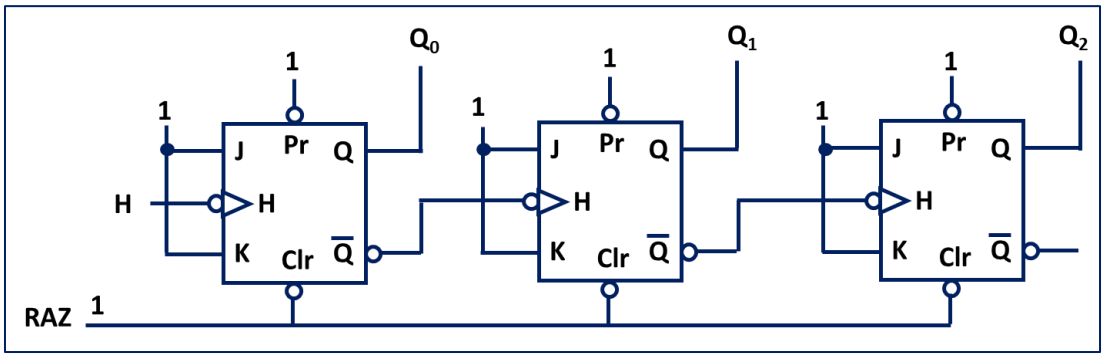


Figure 5.4 : Décompteur asynchrone modulo 8

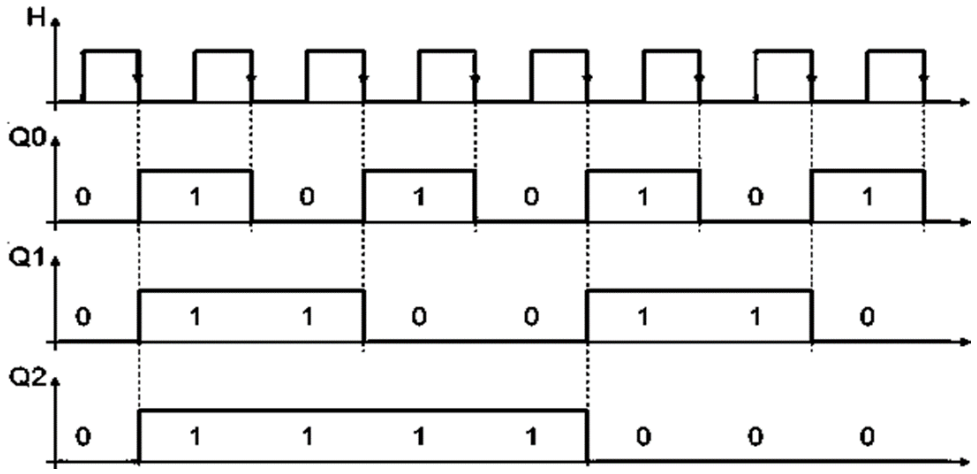


Figure 5.5 : Chronogramme d'un décompteur asynchrone modulo 8

1-2-3 Compteurs asynchrones modulo < 2^n (cycle incomplet)

Dans ce type de compteurs, il faut interrompre le cycle de comptage en réinitialisant les bascules dès la détection du modulo. Dans ce cas, on utilise les entrées de forçage asynchrones des bascules pour la réinitialisation.

Exemple: Compteur asynchrone modulo 6

Dans ce compteur, on utilise comme exemple des bascules JK actives sur front descendant avec des entrées de forçage asynchrones actives sur niveau bas. Dès que le circuit détecte le modulo $6 = Q_2Q_1Q_0 = 110$, il remet à zéro les deux dernières bascules en activant leur entrée CLR (clear) par utilisation d'une porte NAND : $\overline{Q_2Q_1} = 0$. Dans ce cas, le compteur revient rapidement à l'état initial.

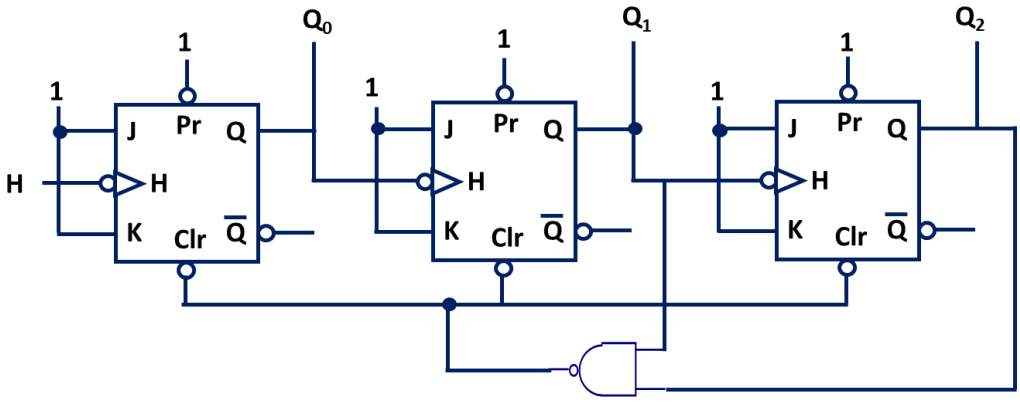


Figure 5.6 : Compteur asynchrone modulo 6

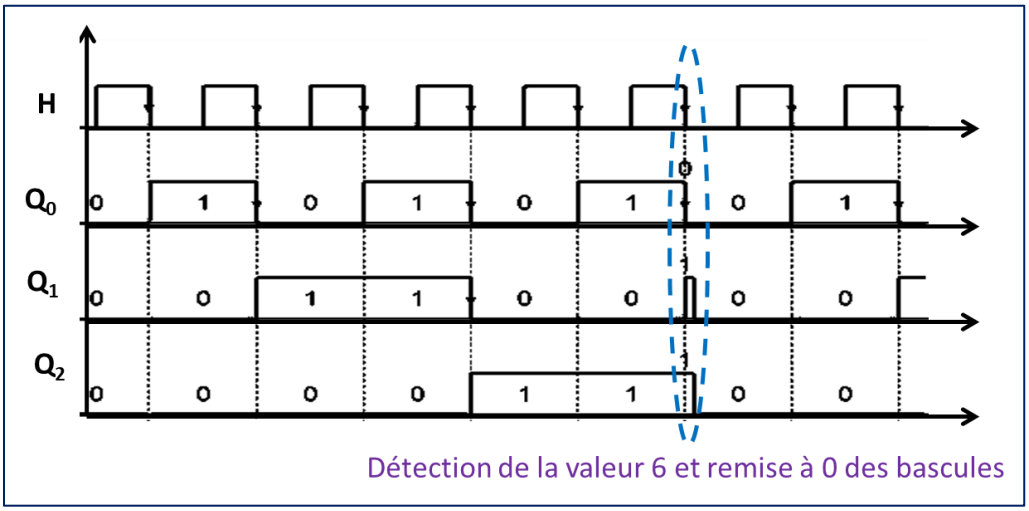


Figure 5.7 : Chronogramme d'un compteur asynchrone modulo 6

1-3 Compteurs synchrones

Dans un compteur synchrone, toutes les bascules sont synchronisées par le même signal d'horloge. Ce compteur peut réaliser n'importe quel cycle de comptage.
Pour faire la synthèse d'un compteur synchrone, il faut déterminer les équations logiques des entrées de chaque bascule en utilisant sa table de transition.

Table de transition des bascules JK et D :

Bascule JK

Q_n	Q_{n+1}	J_n	K_n
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Bascule D

Q_n	Q_{n+1}	D_n
0	0	0
0	1	1
1	0	0
1	1	1

Table 5.1 : Table de transition des bascules JK et D

Exemple de réalisation d'un compteur synchrone modulo 5 par utilisation des bascules JK:

Le nombre de bascules nécessaires est 3 car $5 < 8 = 2^3$.

Le compteur réalise le cycle 0-1-2-3-4-0. Pour déterminer les équations qui commandent les entrées J_i et K_i, on utilise la table de transition de la bascule JK. Le tableau ci-dessous donne les différentes transitions ainsi que les valeurs de chaque entrée J et K.

	Etat présent			Etat future			Entrées des bascules					
	Q2	Q1	Q0	Q2+	Q1+	Q0+	J2	K2	J1	K1	J0	K0
0	0	0	0	0	0	1	0	X	0	X	1	X
1	0	0	1	0	1	0	0	X	1	X	X	1
2	0	1	0	0	1	1	0	X	X	0	1	X
3	0	1	1	1	0	0	1	X	X	1	X	1
4	1	0	0	0	0	0	X	1	0	X	0	X

Table 5.2 : Table d'états présents et futurs et les entrées des bascules JK

Equations logiques des entrées :

J0

Q0 \ Q2Q1	00	01	11	10
0	1	1	X	0
1	X	X	X	X

K0

Q0 \ Q2Q1	00	01	11	10
0	X	X	X	X
1	1	1	X	X

Bascule 0

$$\begin{cases} J_0 = \overline{Q_2} \\ K_0 = 1 \end{cases}$$

J1

Q0 \ Q2Q1	00	01	11	10
0	0	X	X	0
1	1	X	X	X

K1

Q0 \ Q2Q1	00	01	11	10
0	X	0	X	X
1	X	1	X	X

Bascule 1

$$J_1 = K_1 = Q_0$$

J2

Q0 \ Q2Q1	00	01	11	10
0	0	0	X	X
1	0	1	X	X

K2

Q0 \ Q2Q1	00	01	11	10
0	X	X	X	1
1	X	X	X	X

Bascule 2

$$\begin{cases} J_2 = Q_0 Q_1 \\ K_2 = 1 \end{cases}$$

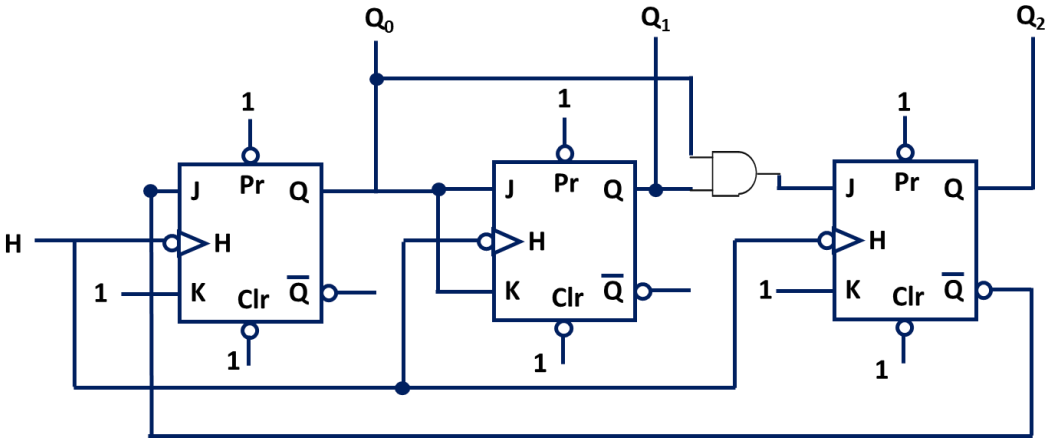


Figure 5.8 : Compteur synchrone modulo 5 par bascules JK

Exemple de réalisation d'un compteur synchrone modulo 5 par utilisation des bascules D:

Le tableau ci-dessous donne les différentes transitions ainsi que les valeurs de chaque entrée D.

	Etat présent			Etat future			Entrées des bascules		
	Q2	Q1	Q0	Q2+	Q1+	Q0+	D2	D1	D0
0	0	0	0	0	0	1	0	0	1
1	0	0	1	0	1	0	0	1	0
2	0	1	0	0	1	1	0	1	1
3	0	1	1	1	0	0	1	0	0
4	1	0	0	0	0	0	0	0	0

Table 5.3 : Table d'états présents et futurs et les entrées des bascules D

Equations logiques des entrées :

D0

Q0	00	01	11	10
0	1	1	X	0
1	0	0	X	X

Bascule 0

$$D_0 = \overline{Q_0}\,\overline{Q_2}$$

D1

Q2Q1 / Q0	00	01	11	10
0	0	1	X	0
1	1	0	X	X

Bascule 1

$$D_1 = \overline{Q_0}Q_1 + Q_0\overline{Q_1} = Q_0 \oplus Q_1$$

D2

Q2Q1 / Q0	00	01	11	10
0	0	0	X	0
1	0	1	X	X

Bascule 2

$$D_2 = Q_0Q_1$$

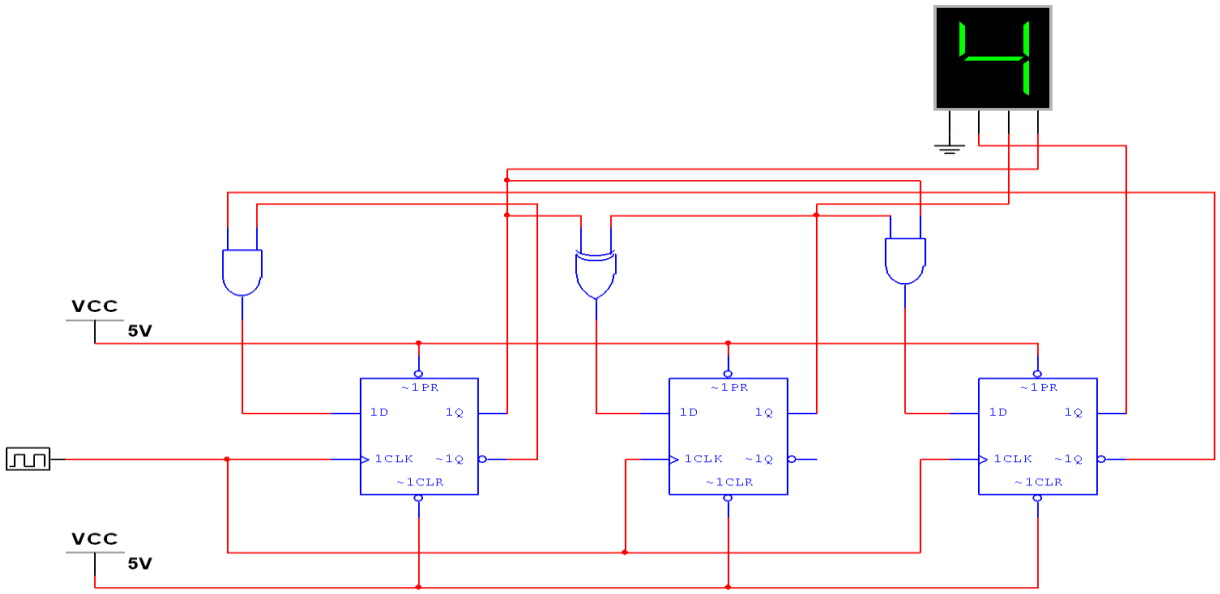


Figure 5.9 : Compteur synchrone modulo 5 par bascules D

2- Registres

2-1 Définitions

Un registre est un ensemble de n bascules (mémoire élémentaire) en série synchronisées par le même signal d'horloge. Ce circuit permet de mémorise un mot binaire de n bits. En outre, les informations binaires sont chargées dans le registre en série ou en parallèle et elles sont lues en série ou en parallèle.

Parmi les applications des registres, on cite :

- Conversion série parallèle des informations numériques
- Mémoires
- Lignes de retard
- Opérations de multiplication et de division par deux

2-2 Registre de mémorisation

Ce type de registre permet la mémorisation d'un mot binaire de n bits (n bascules). L'information est stockée par les bascules en utilisant un signal d'écriture W (Write) connecté à l'horloge de chaque bascule et elles sont lues sur les sorties des bascules en utilisant un signal de lecture R (Read).

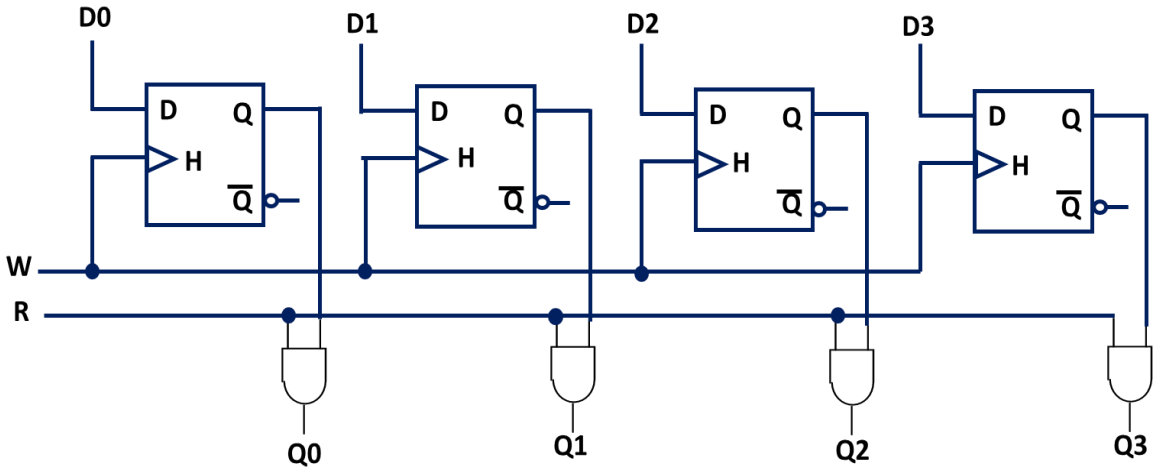


Figure 5.10 : Exemple d'un registre de mémorisation à 4 bits

2-3 Registres à décalage

2-3-1 Registre entrée série- sortie série ou sortie parallèle

L'information est chargée par l'entrée série ES séquentiellement bit après bit après chaque impulsion d'horloge et elle est décalée à droite vers la sortie série SS. Ce registre permet aussi de lire une information en parallèle.

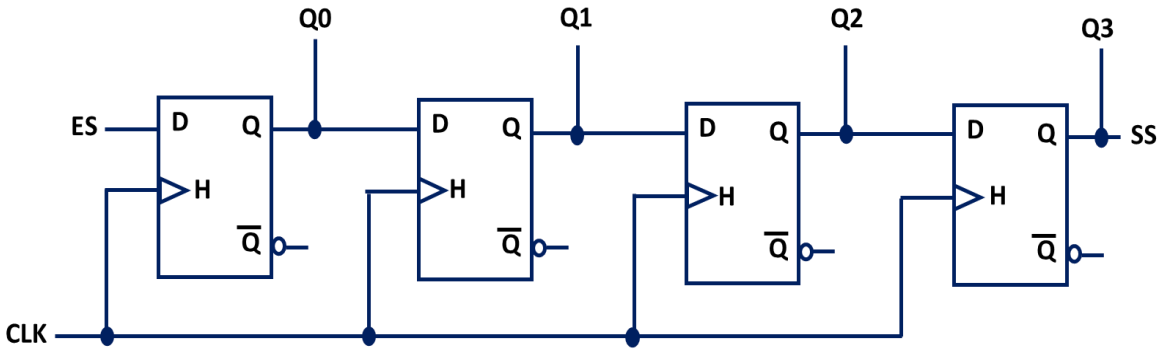


Figure 5.11 : Exemple d'un registre à décalage série-série à 4 bits

2-3-1 Registre à décalage entrée série- sortie série

Ce registre permet de décaler les bits de la droite vers la gauche. Les informations sont appliquées à la dernière bascule à droite.

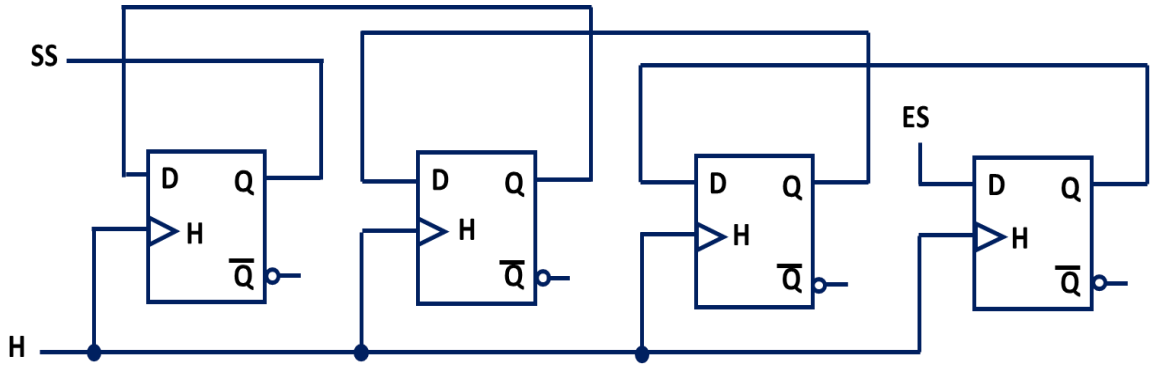


Figure 5.12 : Exemple d'un registre à décalage à gauche série-série à 4 bits

2-3-3 Registre à décalage entrée parallèle- sortie série

Dans ce type de registre, les bits sont appliqués en parallèle à l'entrée de chaque bascule puis ils sont décalés à droite vers la sortie série SS.

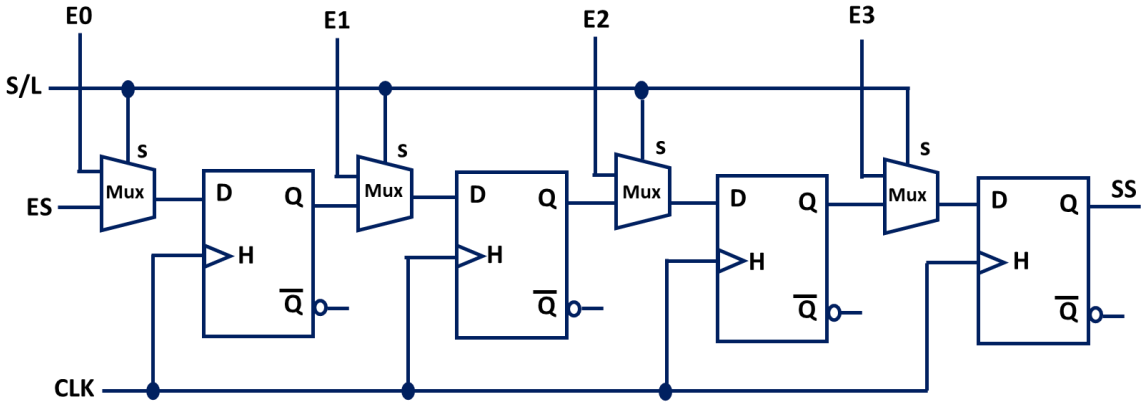


Figure 5.13 : Exemple d'un registre à décalage parallèle-série à 4 bits